Related-work review for SoundCode

Compiled 2026-05-23. A single comparative-reference document for the SoundCode project — to be used as a paste-into-prose source when writing related-work sections.

Reading guide

SoundCode is an LSP-supervised, asynchronous, speculative-decoding-style framework for Rust LLM code generation. A producer LLM streams tokens; a verifier (`cargo check` + rust-analyzer LSP) runs **asynchronously** at structural boundaries (depth-0 `;` and `}` in Rust); on a blocking diagnostic, the system performs a **structural rollback** to the last verified checkpoint and resumes. The async design is forced by `cargo check`'s ~100 ms–few-second cost — Python/C++ verifiers are too fast to bother with async; Rust is in the sweet spot.

The review is organized in two tiers:

- **Inner ring (Section 1)** — 13 papers that are direct comparators in the SoundCode pitch, treated in depth (8-section template: TL;DR / notations / definitions / equations / algorithm / workflow / dataset format / result / SoundCode delta). Subgrouped by design role: rollback-supporting decoders (1.A), non-rollback decoders (1.B), the speculative-decoding structural blueprint (1.C), and the methodological cudgel that every repair paper must address (1.D).
- **Outer ring (Sections 2–5)** — 30 papers from adjacent families (iterative repair, benchmarks, model reports, methodology critiques) treated as fast cards (~6 lines: TL;DR / key mechanism / eval headline / SoundCode delta).

Each section opens with a cross-paper comparison table — the synthesis layer designed for at-a-glance use.

Section 6 collects the SoundCode positioning paragraphs — one per category — drafted to be paste-ready into the related-work section of the eventual paper.

Table of contents

---

1. Inner ring — deep treatment

The thirteen papers below are the ones SoundCode's pitch explicitly positions against — either as a direct sibling (1.A), the closest prior art on the same verifier-on-critical-path axis (1.B), the structural blueprint SoundCode borrows (1.C), or the methodological bar any rollback paper must clear (1.D). Each entry follows an identical 8-section template so cross-paper comparison is straightforward.

1.0 Inner-ring comparison table

Paper	Verifier	Granularity	Async?	Rollback?	Target lang	Headline result	SoundCode delta
ROCODE (2024)	compiler / static analyzer	per-statement	no (sync)	yes (decayed-penalty regen)	Python, C++	HumanEval-ET PassRate 57.3 vs 46.3 (CodeLlama-7B, nucleus p=0.9)	SoundCode is async; Rust; structural-boundary (not state-ment) rollback
SemGuard (2025)	learned 1.3B semantic evaluator	per-line	no (sync)	yes (penalty + N-resample)	Python, Java	SemDiff Pass@1 38.06 vs ROCODE 35.83 (DeepSeek-Coder-6.7B)	SoundCode uses a sound com-piler/LSP, not a learned classifier; no SemDiff training
IterGen (2025)	user predicate over grammar symbols	per CFG non-terminal	no (sync)	yes (KV-cache back-ward)	SQL, Vega-Lite, regex	Spider exec-acc 75.84 vs SynCode 63.72 (9-LLM avg)	SoundCode's verifier is `cargo check`/LSP, not user predi-cates; target is Turing-complete Rust

Paper	Verifier	Granularity	Async?	Rollback?	Target lang	Headline result	SoundCode delta
MGD (2023)	LSP completions	per-token at triggers	no (sync)	**no** (prevention)	Java	DOTPROMPT CR 73.0 vs text-davinci-003 62.7 (SantaCoder-1.1B+MGD)	SoundCode is *recovery* not prevention; structural boundary not per-token
Type-Constrained (2025)	formal type system	per-token	no (sync)	**no** (rejection-mask)	TypeScript	HumanEval-TS pass@1 43.4 (+5.7%) vs vanilla 41.0 (CodeLlama-34B)	SoundCode is async, structural-boundary, with rollback rather than per-token rejection
PICARD (2021)	hand-written incremental parser	per-token (beam)	no (sync)	**no** (rerank)	SQL	Spider EM 71.9 / EX 75.1 (T5-3B + PICARD)	SoundCode's verifier is a real compiler not a parser; rollback not rerank
Synchromesh (2022)	hand-coded DSL completion engine	per-token	no (sync)	**no** (prevention)	SQL, Vega-Lite, SM-CalFlow	Spider EX 64% (Codex 175B + CSD + TST)	SoundCode = real compiler instead of per-DSL CE; async; Turing-complete target
GCD (2023)	CFG / EBNF grammar	per-token	no (sync)	**no** (mask)	Structured NLP	cIE F1 36.0 (LLaMA-33B+GCD) > GenIE-T5-base 34.8	SoundCode = semantic verifier, not syntactic CFG

Paper	Verifier	Granularity	Async?	Rollback?	Target lang	Headline result	SoundCode delta
DOMINO (2024)	grammar + tokenizer-aligned prefix trees	per-token (opportunistic)	no (sync)	**no** (mask)	Grammar tasks (JSON, etc.)	GSM8K JSON 1.77× throughput, 0.418 acc (Mistral-7B)	SoundCode = semantic not syntactic; complements DOMINO as a cheap front-end
Speculative Decoding (2023)	target LLM (probabilistic)	per-token	**yes**	yes (per-token)	text/code	T5-XXL EnDe 3.4× speedup, identical output	SoundCode = *semantic* verifier; structural-boundary rollback
Speculative Sampling (2023)	target LLM (probabilistic)	per-token	**yes**	yes (per-token)	text/code	Chinchilla 70B HumanEval 2.46× speedup	Same comment as Leviathan — DeepMind variant
Lookahead Decoding (2024)	same LLM via Jacobi	per-token	**yes**	yes (n-gram trie)	text/code	LLaMA-2-Chat 7B MT-Bench 1.88×; CodeLlama 8-GPU ClassEval 3.78×	SoundCode = external semantic verifier; structural not n-gram

Paper	Verifier	Granularity	Async?	Rollback?	Target lang	Headline result	SoundCode delta
Olausson (2024)	unit tests + LLM critique	whole program	n/a	n/a (whole regen)	Python	APPS GPT-4 self-repair ×1.08 vs matched i.i.d. (no decisive win when M_F = M_P)	SoundCode's verifier is *strictly* stronger than the LLM on type/borrow checking — falls into the regime where Olausson predicts repair *does* beat i.i.d.

Two cross-cutting observations from the table: (i) every paper in 1.A–1.B is **synchronous** — the entire async axis SoundCode occupies is unaddressed by prior decoding-time work; (ii) every paper in 1.C uses the LLM itself as verifier, so the **semantic-verifier × structural-rollback × async** combination is genuinely the open design point.

1.A Decoding-time methods *with* rollback

The three sibling papers. ROCODE and SemGuard target the exact same design problem SoundCode does — error accumulation during autoregressive code generation — and resolve it by rolling back mid-stream. IterGen does the same for structured-output DSLs.

ROCODE: Integrating Backtracking Mechanism and Program Analysis in Large Language Models for Code Generation (Jiang et al., 2024) **TL;DR.** LLM code generation is auto-regressive, so a single bad token cascades into accumulated errors that post-hoc revision struggles to fix. ROCODE runs the compiler as an incremental program analyzer on each new statement, and on a detected error rolls the generation back to either the error line or — when the root cause is upstream — the highest-entropy token in the recent prefix, then re-decodes under exponentially-decaying penalties on the offending token paths recorded in a Trie. On HumanEval with CodeLlama-7B under nucleus sampling ($p = 0.9$), it lifts PassRate from 26.7 to 57.3 (+30.6) and reaches 99.1% compiler pass rate, beating the best prior baseline by 23.8 points in PassRate while cutting tokens 19.3% vs. post-revising.

Notations.

Symbol	Meaning
x	natural-language requirement (prompt)
y	generated code
$\mathcal{M}$	the LLM (decoder)
$\mathcal{C}$	program-analysis tool (a compiler)
s_i	i-th statement generated by $\mathcal{M}$

Symbol	Meaning
e_i	error report for s_i: $\{\text{result}, \text{type}, \text{lineno}, \text{offset}\}$
H_t	Shannon entropy of $\mathcal{M}$'s distribution at position t
r	rollback point $= [\text{lineno}, \text{offset}]$
λ	decay factor for the regeneration penalty, $\lambda \in (0, 1)$
$T = (U, E)$	Trie tree of token nodes recording all attempted generation paths

Definitions. $\mathcal{M}$ is a frozen auto-regressive LLM that emits one statement s_i per step conditioned on x and the concatenation $S_{:i-1} = s_0 \| \cdots \| s_{i-1}$. $\mathcal{C}$ is a compiler invoked incrementally on $S_{:i-1} \| s_i$; it returns e_i whose `result` is `success` or `failure` and, on failure, the error `type` plus a `(lineno, offset)` source location. A rollback point r is the source position the generator will reset to before resuming. The entropy H_t over vocabulary V measures the model's local uncertainty and is used as a secondary rollback signal when the compiler-reported location is downstream of the true root cause. A Trie tree T has nodes U that are emitted tokens and edges E giving prefix context; each completed attempt corresponds to a root-to-leaf path, and erroneous suffixes accrue penalties carried into future re-decodes. The decay factor λ controls how steeply the per-token penalty PN drops off as one moves backward from the error token to r.

Equations.

$$s_i = \mathcal{M}(x, S_{:i-1}), \qquad e_i = \mathcal{C}(S_{:i-1} \| s_i) \tag{1,2}$$

Generate the next statement, then analyze the cumulative program.

$$e_i = \{\text{result}, \text{type}, \text{lineno}, \text{offset}\} \tag{3}$$

Structured diagnostic returned by the compiler.

$$r_e = [e.\text{lineno}, e.\text{offset}] \tag{4}$$

First-try rollback point: the error site itself.

$$H_t = -\sum_{j=1}^{|V|} p(y_t = v_j \mid y_{<t}, x) \log p(y_t = v_j \mid y_{<t}, x) \tag{5}$$

Per-position entropy of the next-token distribution.

$$t^* = \arg\max_{t \in [0, |y|]} H_t, \qquad r_h = [\text{ConvertToLineno}(t^*, y), 0] \tag{6,7}$$

If r_e fails to fix the recurrence, fall back to the statement containing the most uncertain token.

$$\text{PN}(v \mid y_{<t}) = \begin{cases} \lambda^{t-r} & v = y_t \\ 1 & \text{otherwise} \end{cases} \tag{8}$$

Penalty decays geometrically with distance from the rollback point back to the offending token.

$$p_c(y'_t \mid y_{<t}) = \frac{p(y'_t \mid y_{<t}) \cdot \mathrm{PN}(y'_t \mid y_{<t})}{\sum_v p(v \mid y_{<t}) \cdot \mathrm{PN}(v \mid y_{<t})} \tag{9}$$

Renormalized constrained distribution used for sampling after rollback.

Algorithm.

- I/O tuple: `input = (x: requirement, M: LLM); output = (y: code)`
- Pseudocode (Algorithm 2, faithful):

```
Initialize Trie T = empty, i = 0
s_i <- M(x); T.update_stmt(s_i)
while s_i does not contain EOS:
    # Incremental Error Detection
    e_i <- C(T.stmts)                       # Eq. (2)
    T.update_report(e_i)
    # Strategic Rollback
    if e_i.result == 'failure':
       r <- RollBack(T.stmts, T.reports)  # Alg. 1: try r_e, else r_h via Eq. (5,6,7)
        T.rollback_to(r)
    # Constraint Regeneration
    T.update_pn(T.stmts, r)                 # Eq. (8): lambda^{t-r} penalty
    s_{i+1} <- M(x, T.stmts, T.pn)          # Eq. (9): renormalized sampling
    i <- i + 1; T.update_stmt(s_{i+1})
return T.get_final_gen_code()
```

`RollBack` first asserts e_n.result = failure; if e_n.lineno = e_{n-1}.lineno (recurrence) it falls back to r_h at the highest-entropy statement, else to r_e at the error line. There is also an ADSL-based repeat-pattern detector that flags consecutive identical statement types as failures.

Workflow.

```
prompt x
  -> M emits statement s_i (token-by-token, appended to Trie path)
      -> compiler C runs on S_{:i-1} || s_i  [SYNCHRONOUS, every statement boundary]
            -> success -> continue to s_{i+1}
            -> failure -> RollBack:
                 if same line as previous error -> r_h (max-entropy statement)
               else                             -> r_e (compiler-reported line/offset)
                 -> mark erroneous suffix in Trie with decaying penalty PN (Eq. 8)
                 -> resume decoding under constrained distribution p_c (Eq. 9)
  -> EOS or token budget -> run full hidden test cases for final verification
```

Dataset format. `(prompt: str, public_test_cases: List[(stdin, stdout)], private_test_cases: List[(stdin, stdout)], language: {Python, C++})`. Public cases (when available) drive the runtime-error checks during generation; private cases score PassRate at the end.

Result. On HumanEval(ET)/MBPP(ET)/CodeForces2305 with CodeLlama-7B (and 34B for CodeForces), ROCODE outperforms 9 baselines (3 sampling, 2 sampling-then-filter/post-revise, plus PG-TD, MBR-EXEC, MGD, AdaPT) across PassRate, AvgPassRate, and CCP. Headline on HumanEval(ET) nucleus $p = 0.9$: PassRate 57.3 vs. PG-TD 46.3 (best prior) and base nucleus 26.7; CCP reaches 99.1%. Multilingual: HumanEval-CPP PassRate 39.6 vs. best baseline 29.5 (+34.2% relative). Cost: 503.1 tokens vs. 623.4 for post-revising

(-19.3%) and 675.2 for PG-TD; runtime 0.622 min — slower than vanilla sampling but faster than PG-TD. Ablation (Table IV) shows program-analysis-based detection beats entropy-only detection (57.3 vs 45.1 PassRate), $r_e + r_h$ beats single rollback strategies, and the constraint-regeneration penalty matters (vs. constraint-free resampling 50.7).

Benchmark	Model	Metric	Their result	Best prior baseline
HumanEval(ET), nucleus $p = 0.9$	CodeLlama-7B	PassRate	57.3	46.3 (PG-TD)

SoundCode delta. - **Same as SoundCode:** statement-granularity verification using a real compiler/static analyzer; structural rollback to a checkpoint on a blocking diagnostic; resume decoding from that checkpoint instead of restarting; model-agnostic, training-free; Trie-style memory of failed attempts to avoid re-emitting the same erroneous suffix. - **Different:** ROCODE is **synchronous** — the compiler is invoked between statements and the decoder blocks while $\mathcal{C}$ runs; SoundCode is **asynchronous**, exploiting cargo+rust-analyzer's ~100ms–few-sec latency to keep token streaming live. ROCODE targets Python/C++ where compiler calls are cheap; SoundCode targets Rust specifically because the verifier cost is in the async sweet spot. ROCODE's rollback point comes from compiler line/offset plus a fallback to the highest-entropy token; SoundCode's checkpoints are syntactic (depth-0 ;/}), not entropy-driven. ROCODE adds a soft probabilistic penalty λ^{t-r} at retry, whereas SoundCode performs hard structural rollback without penalty re-weighting. ROCODE also uses ADSL-based repeat-pattern detection orthogonal to the compiler signal. - **Implication for SoundCode design:** (i) ROCODE quantifies that statement-granularity beats post-revising in both quality (+pass) and tokens (−19.3%) — supports the structural-checkpoint thesis. (ii) The "recurring error → jump to upstream high-entropy token" heuristic is a useful augmentation: cargo errors in Rust frequently report symptoms (e.g., move/borrow failure) far from their root cause; an entropy-based secondary checkpoint could complement SoundCode's purely syntactic rollback. (iii) ROCODE's λ-decayed penalty is a candidate to layer on top of SoundCode's hard rollback to discourage immediate re-emission of the same erroneous prefix when async detection lands. (iv) ROCODE explicitly does not run async — confirming SoundCode's "Rust is the sweet spot" framing and the lack of prior async LSP-supervised decoders to compare against.

SemGuard: Real-Time Semantic Evaluator for Correcting LLM-Generated Code (Wang et al., 2025) **TL;DR.** Semantic errors — code that compiles but behaves incorrectly — account for >60% of faults in LLM-generated code, and post-hoc repair (e.g., ROCODE) detects them only after full execution and mis-localizes the faulty line via entropy heuristics. SemGuard fine-tunes a small (1.3B) LLM-based binary classifier on a new line-level diff corpus (*SemDiff*) and embeds it in the decoder so that, after every emitted line, it scores the partial program and rolls back to the offending line if the score drops below 0.5. SemGuard-Penalty lowers semantic-error rate by **19.86%** vs. ROCODE on *SemDiff* and lifts Pass@1 by **48.92%** on *LiveCodeBench* with CodeLlama-7B, while using **31% fewer tokens** and **60% less time** than ROCODE.

Notations.

Symbol	Meaning
$C = \{l_1, \ldots, l_n\}$	Code as an ordered sequence of lines l_i
$J(C_{\text{corr}}, C_{\text{err}})$	Jaccard similarity between correct/erroneous n-gram sets

Symbol	Meaning
D, i^*	Set of differing line indices; $i^* = \min D$ is the first-deviation line
$S = \langle l_1, \dots, l_n \rangle$	Input line sequence to the evaluator
$p = \sigma(W V_{\text{CLS}} + b) \in [0, 1]$	Predicted correctness probability of a code fragment
$s_t \in [0, 1]$	Evaluator confidence after the t-th generated line
$\lambda \in (0, 1)$, N	Token-penalty factor and max resample attempts per line

Definitions. A *line* l_i is one source-code line; a *prefix* $L_{1:t}$ is the first t generated lines. *SemDiff* pairs a correct submission C_{corr} and an incorrect one C_{err} from CodeNet, kept only when their n-gram Jaccard similarity $J > 0.9$ so they differ in a small index set D; the first-deviation line $i^* = \min D$ becomes the supervision cut-point. The *evaluator* is a frozen backbone LLM (DeepSeek-Coder-1.3B) plus a trainable linear head over the CLS/BOS embedding $V_{\text{CLS}} \in \mathbb{R}^d$, returning correctness probability p via sigmoid σ. At decoding the evaluator emits s_t per line; $s_t < 0.5$ flags semantic drift. *SemGuard-Penalty* additionally multiplies the next-token distribution's probability of the just-generated token by λ to discourage repeating the same mistake, then resamples up to N times. Pass@1 is unbiased Pass@k at $k = 1$.

Equations. Numbered as in paper.

$$J(C_{\text{corr}}, C_{\text{err}}) = \frac{|T_{\text{corr}} \cap T_{\text{err}}|}{|T_{\text{corr}} \cup T_{\text{err}}|} \tag{1}$$

Lexical near-duplication filter on candidate pairs (retain $J > 0.9$).

$$D = \{\, i \mid l_i^{\text{corr}} \neq l_i^{\text{err}} \,\} \tag{2}$$

Set of line indices that differ between the matched correct/erroneous programs.

$$S_{\text{corr}} = \langle l_1^{\text{corr}}, \dots, l_{i^*}^{\text{corr}} \rangle, \quad S_{\text{err}} = \langle l_1^{\text{err}}, \dots, l_{i^*}^{\text{err}} \rangle \tag{3}$$

Minimal-contrast training prefixes differing only at line i^*.

$$\mathcal{L} = -\frac{1}{k} \sum_{i=1}^{k} \big(y_i \log p_i + (1 - y_i) \log(1 - p_i) \big) \tag{4}$$

Standard binary cross-entropy on k training fragments with label $y_i \in \{0, 1\}$.

$$p_i' = \lambda p_k \text{ if } i = k, \text{ else } p_i; \qquad p_i'' = \frac{p_i'}{\sum_j p_j'} \tag{5}$$

Down-weight the just-emitted faulty token by λ, then renormalize.

$$L_t^\star = \arg \max_{j \in \{1, \dots, N\}} s_t^{(j)} \tag{6}$$

After N resampled lines without crossing 0.5, keep the line with highest evaluator score.

$$\text{Pass@}k = \mathbb{E}_{\text{Problems}}\left[1 - \frac{\binom{n-c}{k}}{\binom{n}{k}}\right] \tag{7}$$

Unbiased Pass@k from c passing samples out of n.

Algorithm.

- I/O tuple: `input = (problem_prompt: str, generator_LLM, evaluator_LLM, lambda: float, N: int, threshold=0.5); output = (final_code: str)`
- Pseudocode (line-level decoding with evaluator-guided rollback + token penalty):

```
prefix L = []                                    # accepted lines so far
t = 1
while not END_OF_PROGRAM:
    for attempt j in 1..N:
        line_j = generator_LLM.sample_line(prompt, L)   # standard AR until newline
        s_j    = evaluator_LLM.score(prompt, L + [line_j])
        if s_j > 0.5:
            L.append(line_j); break                     # accept
        # else: penalize first non-indented token of line_j
        k = first_non_indent_token_index(line_j)
        p = generator_LLM.next_token_dist_at(L, line_j[:k])
        p[k] *= lambda                                  # Eq. (5)
        p = renormalize(p)
        generator_LLM.set_dist_override(p)              # resample line_j
    else:
        # No attempt cleared threshold -> keep best (Eq. 6)
        L.append(argmax_j line_j by s_j)
    t += 1
return join(L)
```

Workflow. Text flowchart of runtime control flow.

```
prompt
  -> generator emits line l_t (token-by-token)
       -> on newline, freeze L_{1:t}
            -> evaluator(prompt, L_{1:t}) -> s_t in [0,1]
                 -> if s_t > 0.5:
                       -> accept l_t, advance t
                 -> else (rollback to start of l_t):
                       -> penalize 1st non-indent token by lambda
                            -> renormalize, resample l_t (attempt j+1)
                                 -> repeat up to N attempts
                                      -> if all fail: keep argmax_j s_t^{(j)}
  -> continue until EOS
```

Dataset format. Training fragment: `(prefix_lines: List[str], label: int in {0,1})` derived from `(problem: str, C_corr: List[str], C_err: List[str], i_star: int)` where labels mark a prefix as correct (truncated C_{corr} at i^*) or incorrect (truncated C_{err} at i^*). Source: CodeNet pairs filtered by Jaccard > 0.9; LLM-assisted localization (DeepSeek-V3) for multi-line diffs.

Result. SemGuard-Penalty achieves Pass@1 = 38.06% with DeepSeek-Coder-6.7B on *SemDiff* vs. ROCODE 35.83% (+2.23 pts; the paper reports a 19.86% relative reduction in semantic error rate). On transfer benchmarks, SemGuard-Penalty wins 7/8 (model, benchmark) cells; on *LiveCodeBench* with CodeLlama-7B it reaches 8.28% vs. ROCODE 8.73% (one loss) but with DeepSeek-Coder-6.7B reaches 10.04 vs. 9.06. On Java (*SemDiff-Java*) it lifts Pass@1 by

15–27% relative across 4 backbones. Key ablation: swapping evaluator from CodeT5-770M to DeepSeek-1.3B raises Pass@1 from 28.33 to 38.06, showing evaluator quality dominates; the token-penalty alone adds ~5 pts over SemGuard-Random. Cost: 175.6 tokens and 12.98 s/task vs. ROCODE's 253.8 / 32.50 s.

Benchmark	Model	Metric	Their result	Best prior baseline
SemDiff	DeepSeek-Coder-6.7B	Pass@1 (%)	38.06	35.83 (ROCODE)

SoundCode delta. - **Same as SoundCode:** line-/structural-boundary-triggered verification of partial code; rollback to the deviation point and resume generation rather than discarding the whole program; targets the semantic gap that compilers alone miss; cost-aware design that constrains how often verification fires. - **Different:** verifier is a *learned* small LLM scoring semantic plausibility, not a deterministic toolchain (cargo check + rust-analyzer); requires a curated diff-pair training set (*SemDiff*); language is Python/Java with no static-analysis ground truth; verification is *synchronous* per line (no async hiding of latency); rollback granularity is one source line, not a structural Rust checkpoint at depth-0 `;/}`; no token-logit masking, only post-hoc penalty and resample; provides no soundness guarantee — false positives are quantified by FPR rather than ruled out. - **Implication for SoundCode design:** (i) Validates that line-granular rollback + post-rollback token penalty is empirically cheaper and more accurate than ROCODE-style whole-program retries — SoundCode should consider a depth-0 statement penalty when re-emitting after a cargo-check failure to suppress oscillation. (ii) SemGuard's evaluator is needed because Python lacks a cheap sound oracle; in Rust, cargo + rust-analyzer *is* that oracle, so SoundCode can skip the heavyweight evaluator-training pipeline and instead invest the same latency budget in *async* LSP calls. (iii) SemGuard's per-task FPR centered ~0.26–0.34 sets a quantitative bar: SoundCode's blocking-diagnostic gate should beat this by leveraging compiler ground truth, not learned heuristics. (iv) SemGuard's failure modes (non-local logic across files/functions) motivate SoundCode's reliance on rust-analyzer's cross-file index, which addresses precisely this gap.

IterGen: Iterative Semantic-aware Structured LLM Generation with Backtracking (Ugare et al., 2025) TL;DR. IterGen is a Python library that wraps grammar-guided LLM decoding with explicit `forward`/`backward`/`view` primitives indexed by grammar symbols (terminals and non-terminals from a Lark/BNF grammar), letting a user program semantically check the partial output and roll back exactly to a chosen symbol boundary instead of restarting generation. The key technical trick is a symbol position map maintained on every shift-reduce LR parse step, plus a tokenizer-aware mapping that crops the KV cache and decoding trace to the corresponding token index. The authors evaluate three case studies: SQL on Spider (+18.5% accuracy over SynCode, averaged across nine LLMs), Enron email-leak reduction on DecodingTrust (51.4% to 0% leakage), and Vega-Lite on NLV (+17.8% validity).

Notations.

Symbol	Meaning
Σ	finite character alphabet; Σ^* all finite strings over Σ
$V \subseteq \Sigma^*$	LLM token vocabulary; $M : V^* \to \mathbb{R}^{\lvert V\rvert}$ is the LLM scoring function
$\mathcal{S}$	set of grammar symbols (terminals and non-terminals) of a context-free grammar G
$C : \Sigma^* \times \mathcal{S} \to \mathbb{I}$	count of occurrences of symbol $S \in \mathcal{S}$ in a partial output (where $\mathbb{I}$ denotes the integers)

Symbol	Meaning
$\mathcal{D} : \mathcal{S}' \to \mathbb{I} \times \mathbb{I}$	symbol position map; each occurrence in the current output maps to a (start, end) character-index pair
$\mathcal{H}$	decoding trace, a tree of generated tokens with metadata (probabilities, child pointers)
KV	LLM key-value attention cache, kept consistent with $\mathcal{H}$ on every forward/backward call
$\gamma \in [0, 1]$	recurrence penalty; previously-decoded token probabilities are multiplied by $(1-\gamma)^\alpha$

Definitions. A grammar G in BNF/EBNF has terminals (concrete tokens) and non-terminals (placeholders expanded by production rules $S \to S_1 \dots S_n$). A shift-reduce LR parser is a bottom-up CFG parser that reads input left-to-right, shifting lexed terminals onto a stack and reducing the stack top to a non-terminal when a production rule's right-hand side matches. Token misalignment is the gap between the LLM's BPE-style vocabulary V and the lexer's terminal alphabet, since one LLM token may straddle multiple grammar terminals or fragments. IterGen handles misalignment dynamically by maintaining $\mathcal{D}$ at every reduce step: when a rule $S \to S_1 \dots S_n$ fires, IterGen sets $\mathcal{D}(S) = (\mathcal{D}(S_1)_l, \mathcal{D}(S_n)_r)$, recursively inheriting positions from already-mapped right-hand-side symbols. Backtracking is structural: the user names a grammar symbol S and a count n, and IterGen rewinds the output, the decoding trace, the KV cache, and the symbol-position map exactly to the $(\text{total} - n)$-th occurrence of S.

Equations. Numbered for the load-bearing operations.

(1) Tokenizer maps a prompt O_0 to a token sequence: $t_1, \dots, t_k = \text{tokenize}(O_0)$; the LLM scores the next token as $\mathcal{S} = M(t_1, \dots, t_k)$, where $\mathcal{S}$ is a logit vector of length $|V|$. Reading: standard autoregressive decoding step, shared with every LLM library.

(2) $\mathit{softmax}(\mathcal{S}_i) = \exp(\mathcal{S}_i) / \sum_j \exp(\mathcal{S}_j)$, applied as $m \odot \mathit{softmax}(\mathcal{S})$ where $m \in \{0, 1\}^{|V|}$ is a constrained-decoding mask and $\odot$ is element-wise product. Reading: SynCode-style grammar mask is composed with the LLM distribution before sampling.

(3) Forward post-condition: $C(O_f, S) - C(O_i, S) = n$, given inputs $S \in \mathcal{S}$ and $n \in \mathbb{I}$, where O_i is the output before the call and $O_f = O_i + \Delta$ is the output after, with suffix $\Delta \in \Sigma^*$. Reading: `forward(S, n)` keeps appending tokens until exactly n new occurrences of symbol S have been parsed (or EOS / max length terminates early).

(4) Backward post-condition: $\hat{O}_b$ is the maximal prefix of O_i such that $O_i = \hat{O}_b + \Delta$ and $C(\Delta, S) = n$; if $C(O_i, S) < n$ the operation falls back to the initial prompt O_0. Reading: `backward(S, n)` deletes the suffix containing the last n occurrences of S and synchronously crops $\mathcal{H}$ and KV.

(5) Symbol-position update on reduce: $\mathcal{D}(S) = (\mathcal{D}(S_1)_l, \mathcal{D}(S_n)_r)$ when the rule $S \to S_1 \dots S_n$ fires. Reading: the position of a reduced non-terminal is inherited from its leftmost and rightmost children, so the map stays exact across arbitrary nesting.

(6) Recurrence penalty: $\text{scores}[t] \leftarrow \text{scores}[t] \cdot (1-\gamma)^\alpha$, where α is the number of times token t has been backtracked. Reading: after each backward, previously chosen tokens are de-prioritized so the LLM explores a different path on the re-roll.

Algorithm.

- I/O tuple: input = (LLM M, tokenizer T, BNF/EBNF grammar G, prompt O_0, user program with semantic check); output = (string O_n consistent with G and the semantic predicate, or partial output after max_iter attempts)
- Pseudocode (Forward + Backward, condensed from Algorithms 1-3, Appendix A.1):

```
function FORWARD(state, stop_symbol, count):
    init_occ = count_occurrences(state.D, stop_symbol)
    while True:
        scores = state.model(state.cur_tokens, state.KV)
        partial = detokenize(state.T, state.cur_tokens)
     state.parser.update(partial, state.D)         # incremental LR shift/reduce; updates D on eve
      m = generate_mask(state.parser)                # grammar mask (SynCode-style)
        scores = m * scores
        for t in state.H.past_tokens():
            scores[t] *= (1 - gamma) ** count(t)          # recurrence penalty
        t_i = decoding_algorithm(scores)                  # greedy / sample
        if t_i == EOS: break
        cur_occ = count_occurrences(state.D, stop_symbol)
       if cur_occ - init_occ >= count or len(state.cur_tokens) > max_tokens: break
        state.cur_tokens.append(t_i); state.H.add(t_i)
    return detokenize(state.T, state.cur_tokens)

function BACKWARD(state, stop_symbol, num):
    total = symbol_position(state.D, stop_symbol)
   char_pos = get_symbol_pos(total - num)           # target character offset in output
    O_m = detokenize(state.T, state.cur_tokens)[:char_pos]
   tok_pos, remainder = find_token_index(state.H, char_pos)  # may split a token straddling the bour
    state.KV = state.KV.crop(tok_pos)                   # discard suffix of KV cache
    state.D = update_position_map(state.D, char_pos)
    state.cur_tokens = update(state.cur_tokens[:tok_pos], remainder)
    return O_m
```

Workflow. Text flowchart of one IterGen session.

```
User program -- start(prompt O_0) --> tokenize, init KV, init decoding trace H, init symbol map D
       |
       v
   forward(S, n)
      |  loop: LLM scores -> grammar mask (LR parser) -> recurrence-penalty reweight
       |         -> sample token -> append to cur_tokens, H, KV
     |      -> incremental parse: every reduce updates D with (start,end) of new symbol
     |      -> stop when n more occurrences of S are reduced, EOS hit, or max_tokens reached
       v
  view(S)  -> read the substring(s) of cur_tokens that grammar-reduced into symbol S
       |
       +-- semantic check passes --> next forward call or finished()
       +-- semantic check fails  --> backward(S, k)
                                  |  look up (start,end) of the k-th-from-last S in D
                            |  slice cur_tokens, crop KV cache to matching token index
                                 |  trim H and D, mark backtracked tokens for penalty
                                      v
                                resume forward (now exploring a different branch)
```

Dataset format. Tuple (natural-language utterance, optional schema or data-frame context, ground-truth structured output). Concretely: Spider gives (NL question, SQL database schema string, ground-truth SQL query) over 1,034 problems split

easy/medium/hard/extra; DecodingTrust Enron extraction gives (`5-shot prompt listing 5 (name, email) pairs and asking for the victim's email, victim email address as ground truth`) over 100 prompts; NLV gives (`NL utterance, dataset name, list of data-field names, ground-truth Vega-Lite JSON spec`) over 814 samples.

Result. SQL on Spider averaged over nine LLMs (Qwen2.5-{0.5B, 0.5B-Instruct, 1.5B, 1.5B-Instruct, Coder-1.5B}, Llama-3.2-{1B, 3B}, Llama-2-7b-chat, Meta-Llama-3-8B): IterGen reaches 41.63% overall accuracy and 75.84% execution success vs. SynCode's 35.22%/63.72% and Standard's 28.90%/50.28% (Table 8). Privacy: zero leaks across all eight tested LLMs vs. 45-67 leaks per 100 prompts for unconstrained Standard, with only a small per-completion time overhead (e.g. Llama-3-8B 0.66s to 0.76s) and an average extra 4-7 tokens regenerated (Table 2). Vega-Lite on NLV: IterGen beats SynCode on every model evaluated (Table 3). Per-paper headline numbers: +18.5% SQL accuracy and +17.8% Vega-Lite accuracy over SynCode, 100% privacy preservation.

Benchmark	Model	Metric	Their result	Best prior baseline
Spider text-to-SQL	9-model average	exec. accuracy / execute%	41.63 / 75.84	SynCode 35.22 / 63.72

SoundCode delta. - **Same as SoundCode:** Speculative-decode-then-verify-then-rollback control flow; verifier runs over fully-parsed structural units rather than every token; rollback reuses the KV cache by cropping rather than re-prefilling; a small per-token "recurrence penalty" prevents the LLM from immediately retracing the rejected path. IterGen's reliance on a shift-reduce LR parser keyed on grammar non-terminals is conceptually the same move as SoundCode's use of depth-0 Rust delimiters (`;`, `}`) as structural checkpoints. - **Different:** IterGen's verifier is whatever Python code the user writes against `view(symbol)` — it has no compiler or LSP in the loop, and the demonstrated checks (schema-name membership, regex email match, JSON field type) are cheap enough to run synchronously between every `forward` call; SoundCode's verifier (cargo check + rust-analyzer) costs ~100 ms to a few seconds and is therefore decoupled asynchronously. IterGen targets declarative DSLs (SQL, JSON/Vega-Lite, email regex) that fit a single Lark grammar; Rust requires whole-crate type inference and borrow checking, which no CFG-level mask captures. IterGen's `backward(S, n)` is parameterized by user-named grammar symbols, not by an external diagnostic's source range. The paper's own limitations section concedes that the recurrence-penalty heuristic skews the LLM distribution at the first retried token. - **Implication for SoundCode design:** Reuse IterGen's symbol-position-map idea as SoundCode's "checkpoint table": instead of mapping grammar symbols to character offsets, map verified structural boundaries (depth-0 `;`/`}` token indices) to (token-index, KV-snapshot, cargo-check-result) triples. The KV-cropping rollback in Algorithm 3 (lines 5–9) is a near-drop-in template for SoundCode's structural rollback, including the awkward straddling-token handling that token-vs-character-boundary mismatch forces. Adopt a recurrence-penalty knob as a guard against oscillation when rollback to the same checkpoint fires repeatedly (this maps to the "prompt history as oscillation guard" finding in week 4 design notes). Finally, IterGen's evaluation pattern — (a) accuracy on a held-out structured-output benchmark, (b) average backward calls per task, (c) wall-clock overhead, (d) avg extra tokens — is exactly the four-axis budget SoundCode should report.

1.B Decoding-time methods *without* rollback

The six papers below sit on the same "verifier on the critical path" axis as SoundCode, but commit instead of recover — they mask or rerank candidates per token to *prevent* invalid output rather than emitting freely and rolling back. The trade-off they bake in: sound prevention at the cost of synchronous overhead at every step, plus a verifier that must be cheap enough to run in the hot loop. SoundCode flips both choices — it accepts speculative tokens between checkpoints and runs the (expensive) verifier asynchronously.

MGD: Guiding Language Models of Code with Global Context using Monitors (Agrawal et al., 2023) TL;DR. MGD (Monitor-Guided Decoding) augments a frozen code LM by attaching a stateful "monitor" that, at user-specified trigger points, invokes a static-analysis Language Server (via LSP) and reshapes the LM's next-token logits with a binary mask that admits only tokens forming a prefix of any type-consistent identifier suggested by the analysis. The flagship instantiation guards object dereferences (the "." trigger in Java) so that the LM can only emit fields/methods declared on the resolved receiver type, including types defined in other repository files or build-time artifacts (Lombok, ProtoBuf). On the authors' new PRAGMATICCODE / DOTPROMPTS benchmark, MGD raises compilation rate and next-identifier match for every tested LM (CodeGen-{350M, 2B, 6B}, SantaCoder-1.1B, text-davinci-003), and lets a 1.1B SantaCoder beat 175B text-davinci-003 on compilation rate and next-identifier match.

Notations.

Symbol	Meaning
L_θ	autoregressive code language model with parameters θ and vocabulary V
$x_1, \ldots, x_n$	partial code (tokens) already generated; x_{n+1} is the candidate next token
p	additional prompt (e.g., FIM suffix), beyond the auto-regressive prefix
C	repository-level context (cross-file source, build artifacts, library bindings)
φ	a property to enforce (e.g., type-consistent dereference); A_φ is the static analysis for φ
$M_\varphi = (A_\varphi, s_0, S, \text{pre}, \text{update}, \text{maskgen})$	monitor: state machine with wait state s_0, state set S, trigger pre-condition, state-update function, and mask generator
ℓ, m	per-token logits from L_θ, and the binary mask over V produced by M_φ

Definitions. A **monitor** is a stateful interface between L_θ and A_φ. It sits in s_0 until the partial code satisfies pre (e.g., last emitted token is "."), at which point A_φ runs on the partial code plus repository context C and returns a set of allowable identifier strings s. While $s \neq \emptyset$, maskgen(s, V) produces a binary mask over the LM vocabulary that admits exactly the tokens which extend the current prefix toward some member of s; once a delimiter symbol from a fixed set E (e.g., "(", ",") is sampled, the monitor returns to s_0. A **type-consistent identifier** for a dereference `obj.` is one declared in the resolved type T of `obj`, possibly defined cross-file or in libraries. The **repository context** C includes other source files, libraries, and intermediate build artifacts; MGD never injects C into the prompt — only the analysis output reshapes logits.

Equations. Decoding under monitor composition $L_\theta \| M_\varphi$, with current monitor state s:

$$(L_\theta \| M_\varphi)(x_{n+1} \mid x_1, \ldots, x_n; C, p, s) = \begin{cases} \text{softmax}(\ell)[x_{n+1}] & \text{if } s = s_0 \\ \text{softmax}(\ell \oplus m)[x_{n+1}] & \text{otherwise} \end{cases} \quad (1)$$

Reading: in the wait state the LM samples freely; otherwise the mask is fused with logits before softmax.

$$\ell = L_\theta(\cdot \mid x_1, \ldots, x_n; p) \quad (2)$$

Reading: ℓ is the standard LM logit vector over V.

$$m = \text{maskgen}(s, V) \quad (3)$$

Reading: the mask is determined by the monitor state s; $m[t] = 1$ iff token $t \in V$ matches the regex $w \cdot E \cdot \Sigma^*$ for some $w \in s$ (string prefix matching with allowed terminators E).

$$s' = \begin{cases} A_\varphi(x_1, \ldots, x_n; C) & \text{if } s = s_0 \wedge \text{pre}(s; x_1, \ldots, x_n) \\ \text{update}(s, x_{n+1}) & \text{otherwise} \end{cases} \quad (4)$$

Reading: state transitions either by firing the static analysis (entering monitored mode) or by pruning the residual suggestion set by the prefix just emitted.

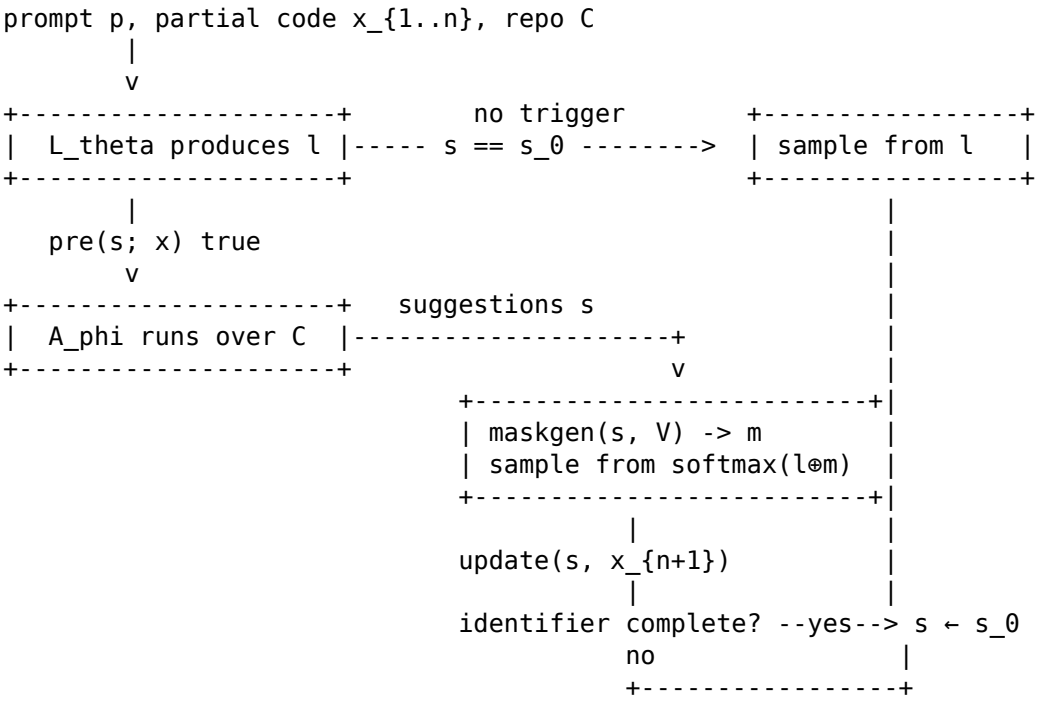
Fusion operator: $(\ell \oplus m)[x] = \ell[x]$ if $m[x] = 1$, else $-K$ for large $K > 0$ (effectively zero probability after softmax).

Algorithm.

- I/O tuple: `input = (L_theta, M_phi, repo C, prompt p, max_len N); output = (x_1, ..., x_T)` with $T \leq N$.
- Pseudocode:

```
s ← s_0; tokens ← []
for step in 1..N:
    l ← L_theta.logits(tokens; p)              # Eq. 2
    if s == s_0:
       if pre(s; tokens):                    # trigger fires, e.g., last token is "."
            s ← A_phi(tokens; C)                 # call LSP / static analyser
            if s == ∅: abort monitoring; s ← s_0
            m ← maskgen(s, V)                    # Eq. 3
            x ← sample(softmax(l ⊕ m))           # Eq. 1, monitored branch
        else:
            x ← sample(softmax(l))               # Eq. 1, wait branch
    else:
        m ← maskgen(s, V)
        x ← sample(softmax(l ⊕ m))
       s ← update(s, x)                      # Eq. 4: prune suggestions by prefix x
        if x contains a symbol from E:           # identifier complete
            s ← s_0
    tokens.append(x)
    if x == EOS: break
return tokens
```

Workflow.



Dataset format. PRAGMATICCODE: tuple (`repository_snapshot`, `build_environment`, `dependency_closure`, `CodeQL_database`) over 100 Java repos. DOTPROMPTS testcase: (`target_method`, `prompt_prefix_up_to_a_dot`, `ground_truth_completion`, `repo_ref`) — 1420 methods, 10538 dereference prompts. MGDMICROBENCH: 10 hand-crafted cases over Java / C# / Rust covering valid-class instantiation, switch-over-enum, argument arity, typestate, and session-type protocols.

Result. With a 6-generation budget on DOTPROMPTS, every model gains compilation rate (CR) and next-identifier match (NIM); SantaCoder-1.1B + MGD surpasses 175B text-davinci-003 on CR and NIM. Mean decoding-time slowdown is 83.16% (CodeGen-6B vs CodeGen-6B-MGD, Table 2).

Benchmark	Model	Metric	Their result	Best prior baseline
DOTPROMPTS (score@6)	SantaCoder-1.1B + MGD	Compilation Rate	73.01 (+21.77%)	text-davinci-003: 62.66

(Same row, NIM: 88.42 for SC-MGD vs 76.94 for text-davinci-003. SC-RLPG-MGD reaches CR 78.14, the strongest non-davinci config.)

SoundCode delta. - **Same as SoundCode:** Treats an LSP / static-analysis signal as a first-class supervisor of a frozen code LM at decode time; uses the same multilspy LSP-client abstraction philosophy (language-agnostic, thin wrapper around language servers including rust-analyzer); both add no training, no architecture change. - **Different:** MGD is a *prevention* mechanism — it masks per-token logits at hand-picked syntactic triggers (e.g., ".") so the model can never emit a bad identifier prefix; SoundCode is a *recovery* mechanism — it lets the producer LLM stream freely, runs `cargo check` + rust-analyzer asynchronously at Rust structural boundaries (depth-0 `;` / `}`), and on a blocking diagnostic performs structural rollback to the last verified checkpoint. MGD is synchronous in the hot loop (every token at a trigger pays an LSP round-trip; reported ~83% slowdown), targets a single property at a

time per monitor (joint monitors are the product construction), and only enforces statically-decidable local constraints (type-consistency, enum membership, argument count, typestate, session types) — it cannot catch semantic errors that surface only to the full type-checker / borrow checker. SoundCode catches arbitrary diagnostics from a real compiler. - **Implication for SoundCode design:** (i) MGD's `pre/update/maskgen` state machine is the right shape for "speculative trigger then verify" — SoundCode's depth-0 `;/}` triggers are exactly the Rust-side analogue of MGD's "." trigger, but applied at the *checkpoint* granularity rather than the *token* granularity, so the LSP round-trip can run async on a stale prefix. (ii) The MGD slowdown number (~83% on a synchronous, per-trigger implementation) is the headline cost SoundCode's async-plus-rollback design must beat to justify itself. (iii) MGD's joint-monitor construction (Section H.2) suggests SoundCode can compose multiple verifiers (cargo check + clippy + rust-analyzer diagnostics) by taking the product of their "blocking diagnostic" predicates rather than picking one. (iv) MGD's failure modes (false positives from imprecise static analysis on partial code) motivate SoundCode's choice to accept bad prefixes and roll back, rather than mask: the verifier only fires on a *complete* structural unit, where the analysis is sound, removing MGD's "what if the analyzer is wrong on a half-typed expression" pathology.

Type-Constrained Code Generation with Language Models (Mündler et al., 2025) **TL;DR.** The authors design a sound constrained-decoding method that enforces well-typedness of LLM-generated code on every sampled token by combining a prefix automaton (which incrementally parses partial programs and tracks type-relevant context) with a type-reachability search (which decides whether a partial expression can be completed to inhabit a required type). The method is formalized on a simply-typed Turing-complete calculus L_B with soundness proofs, then engineered for a substantial subset of TypeScript. On TypeScript HumanEval and MBPP across six 2B-34B open-weight LLMs (Gemma-2 2B/9B/27B, DeepSeek-Coder 33B, CodeLlama 34B, Qwen2.5 32B), type constraining halves compilation errors (74.8% / 56.0% reduction on HumanEval/MBPP synthesis) and raises pass@1 by 3.5%-5.0% on synthesis/translation and 37.0% on repair, with median runtime overhead of 39%/52%.

Notations.

Symbol	Meaning
L, L^p	A language L and its prefix language $L^p := \{s \mid \exists s' : s \circ s' \in L\}$
CE_L	Completion engine: $CE_L(s) =$ true iff $s \in L^p$
$A = \langle \Sigma, Q, \delta, I, F \rangle$	Automaton: alphabet, states, transition function, initial states, accepting states
$\gamma(\mathbf{q}, s)$	Traversal function returning states reachable from $\mathbf{q} \subseteq Q$ after consuming string s
$\Gamma \vdash e : T$	Expression e has type T under type environment Γ (a set of $(x : T)$ bindings)
$A_e \downarrow T$	Expression automaton restricted to expressions inhabiting type T
L_B	The paper's simply-typed Turing-complete core calculus (a subset of safeFTS / TypeScript)

Definitions. A *prefix language* L^p contains every string that is a prefix of some member of L. An automaton A satisfies the *prefix property* iff every reachable state has a path to an accepting state; equivalently $\forall \mathbf{q} \in \gamma(I, s) : \exists s' : \gamma(\mathbf{q}, s') \cap F \neq \emptyset$ (Def. 2). For such a prefix

automaton, $L_r(A) := \{s \mid \gamma(I, s) \neq \emptyset\}$ (the reachable language) equals $L(A)^p$ (Lemma 1), so $CE_{L(A)}(s) := \gamma(I, s) \neq \emptyset$ is a sound, decidable completion engine. *Type reachability* asks whether a parsed expression of derivable type T can be extended by a sequence of operator / call / member-access steps into an expression of goal type G; this is solved by depth-first search over an abstract type graph (nodes = types, edges = well-typed extensions) with cycle marking and a heuristic that prunes branches whose type complexity exceeds that of both source and goal.

Equations. Recursive extension of the expression automaton's transition function (text, p. 11):

$$\forall X, Y : \delta_e(q_Y^X, c) := \begin{cases} \delta_Y(q_Y^X, c) \cup \delta_e(I_{(e)}^X, c) \cup \delta_e(I_{\odot e}^X, c) \cup \delta_e(I_{.n}^X, c) & \text{if } q_Y^X \in F_Y \\ \delta_Y(q_Y^X, c) & \text{otherwise} \end{cases} \quad (1)$$

Reading: while in an accepting state of sub-automaton A_Y (a complete base expression), allow continuing with the same automaton, with grouping `(e)`, with a binary operator extension `\odot e`, or with member access `.n` — otherwise only continue inside A_Y. The superscript X records the string already parsed (used to type the result), Y identifies the active sub-automaton.

Prefix-property characterization (Def. 2):

$$\forall \mathbf{q} \in \gamma(I, s) : \exists s' : \gamma(\mathbf{q}, s') \cap F \neq \emptyset \quad (2)$$

Reading: every state reachable from any initial state by parsing s has at least one continuation s' reaching an accepting state — equivalently $s \in L(A)^p$ implies $s \in L_r(A)$.

Algorithm. - I/O tuple: `input = (current type T of expression e, goal type G); output = (bool: can e be extended to inhabit G?)` - Pseudocode (Algorithm 2, Type Reachability):

```
function REACHABLE(T, G):
  if T == G: return true                          # goal reached
  if T marked: return false else mark T           # cycle break
  for each valid extension step ◇ from T:         # operator, call, or member
    S := type resulting from applying ◇ on T
   if PRUNESEARCH(T, G, S): continue        # heuristic: skip if S more complex than T and G
    if REACHABLE(S, G): return true                # recurse
  return false
```

The outer constrained-decoding loop (Algorithm 1, sample-and-check) repeatedly draws $t \sim v$ from the LLM's next-token distribution, accepts t iff $CE_L(s \circ t)$ holds or $t = \text{EOS}$ with $s \in L$, and otherwise zeroes $v[t]$ and renormalizes — never re-running LLM inference for the same position.

Workflow. Text flowchart:

```
prompt x -> initialize program prefix s = ""
loop until EOS:
  v := LLM(x ∘ s)                                 # one forward pass
  inner loop:
    sample t ~ v
    feed character(s) of t through prefix automaton A:
      A maintains parsed AST + type environment Γ
      at each expression boundary, if a target type T is required:
        call REACHABLE(DERIVABLE(current state), T)
```

```
      if traversal succeeds -> CE_L(s ∘ t) = true -> accept; break
      else -> v[t] := 0; renormalize; resample
  s := s ∘ t
return s   (guaranteed s ∈ L_B, i.e., well-typed)
```

Dataset format. Tuple per benchmark instance: `(natural-language description, TypeScript function header, hidden unit tests)` for *synthesis*; `(Python reference function, TypeScript function header, hidden tests)` for *translation*; `(description, non-compiling TypeScript program, compiler error message, function header, hidden tests)` for *repair*. Benchmarks: TypeScript-translated HumanEval (159 tasks) and MBPP (384 tasks) from MultiPL-E.

Result. Type constraining cuts non-compiling outputs across all six models and both benchmarks (e.g., Gemma 2 9B HumanEval synthesis: 45 -> 13 errors, -71.1%); idealized syntax-only constraining at best removes 9.0%/4.8% of errors on HumanEval/MBPP synthesis vs. 74.8%/56.0% for types. pass@1 rises consistently (avg +3.5% synthesis, +5.0% translation, +37.0% repair); median per-instance overhead 6.7-11.7 s on HumanEval and 4.9-11.7 s on MBPP (+39.1% / +52.1% over vanilla). In 99.4% of decoding steps a single sample passes the check (long-tail loop distribution), so the LLM is only re-queried once per token.

Benchmark	Model	Metric	Their result	Best prior baseline
HumanEval (TS) synthesis	CodeLlama 34B	pass@1	43.4 (+5.7%)	41.0 (Vanilla); 41.0 (idealized Syntax)

SoundCode delta. - **Same as SoundCode:** Uses a per-token verifier of structural / type correctness on the critical decoding path, with hard rollback of any non-conforming sampled token (forcing resample without re-running LLM inference); maintains an evolving program-level abstract state (parsed AST + type environment Γ) so verification is incremental rather than re-parsing from scratch; targets the same failure mode (LLMs hallucinating ill-typed code) and the same task family (synthesis / translation / repair) on the same HumanEval/MBPP families. - **Different:** Verifier is *synchronous and per-token* (blocks before each appended character), not asynchronous at depth-0 ;/} boundaries; verifier is a custom prefix automaton with a bespoke type-reachability search rather than `cargo check` + rust-analyzer LSP; rollback is *token-level* (zero $v[t]$ and resample at the same position) rather than *structural rollback to the last verified checkpoint*; soundness is formalized at language level ($L(A_M) \subseteq L_B$, well-typedness) rather than empirical "no blocking diagnostic"; target language is TypeScript (with a 11,249-LoC custom completion engine, partial feature coverage) rather than Rust via off-the-shelf compiler/LSP; requires white-box next-token logits, ruling out closed APIs. - **Implication for SoundCode design:** This paper is the strongest existing point on the "verifier on critical path, formally sound" end of the design axis SoundCode trades against. It establishes (i) that per-token type checking is implementable and meaningfully reduces compilation errors (the headline >50% reduction is a target SoundCode should at least match on Rust to justify the LSP path), and (ii) that 39-52% median runtime overhead is the price of synchronous verification on a 2B-34B model — motivating SoundCode's asynchronous-at-boundaries design as a latency-vs-soundness trade. The long-tail loop-iteration histogram (99.4% of tokens accepted on first try) suggests that for Rust+LSP, blocking at every token is wasteful and that boundary-triggered checks are likely to dominate in the common case. The fact that Mündler et al. had to hand-build a 11k-LoC engine to approximate the TypeScript type system — and still omit features — argues strongly for SoundCode's choice to reuse rust-analyzer/cargo as a black-box oracle rather than reimplementing Rust's trait + borrow rules. Finally, the paper's repair result (+37% pass@1) suggests SoundCode should evaluate

a repair-style protocol where LSP diagnostics drive a rollback-then-resample loop, not only single-shot generation.

PICARD: Parsing Incrementally for Constrained Auto-Regressive Decoding from Language Models (Scholak et al., 2021) **TL;DR.** PICARD is a constrained-decoding wrapper that, at each step of beam search over a pre-trained T5 text-to-SQL model, runs a fast incremental parser on each candidate continuation and assigns a score of $-\infty$ to tokens whose partial detokenized output fails lexical or grammatical checks. The parser is built from monadic combinators and supports four progressively stricter modes (off, lexing, parsing without guards, parsing with guards that consult the database schema). With PICARD on top of an unmodified fine-tuned T5-3B, the system reaches state-of-the-art exact-set-match and execution accuracy on Spider and CoSQL without any model architectural change.

Notations.

Symbol	Meaning
k	Number of highest-probability tokens PICARD inspects at each decoding step
V	Vocabulary of sub-word tokens of the language model
h	A beam hypothesis: token sequence with associated log-softmax score
$s_t(v)$	Log-softmax score of token $v \in V$ at step t from the LM head
detok$(h \cdot v)$	Detokenized surface-form string after appending v to h
EM% / EX%	Exact-set-match / execution accuracy on Spider
QM% / IM%	Question-match / interaction-match accuracy on CoSQL

Definitions. - *Lexing mode*: parse the partial detokenized string as a white-space delimited bag of SQL keywords, punctuation, operators, literals, and identifiers; reject mis-spelled keywords and identifiers absent from the supplied schema. - *Parsing without guards*: parse into a partial SQL AST; order of clauses matters; reject ill-formed structures and `tid.cid` / `alias.cid` references whose referent does not exist. - *Parsing with guards*: as above plus eager semantic checks that require each referenced table / alias to actually be in scope (i.e., bound in the `FROM` clause) and resolvable to a column cid. - *Finalizing mode*: PICARD is only invoked once the model emits the end-of-sequence token (post-hoc filter), as opposed to *incremental* checking at every step. - *Guards*: side-conditions injected into the partial parse that fail-fast on schema-incompatible compositions before the AST is complete.

Equations. PICARD is algorithmic; the paper states one score-warp rule:

$$\tilde{s}_t(v) = \begin{cases} s_t(v) & v \in \text{Top}_k(s_t) \text{ and PICARD-accepts}(h \cdot v) \\ -\infty & \text{otherwise} \end{cases} \quad (1)$$

Reading: only the top-k tokens are inspected; accepted ones keep their LM score, everything else is masked out, so beam search re-normalizes over the surviving continuations.

Algorithm. - I/O tuple: `input = (hypothesis token ids h, LM log-softmax vector s_t over V, SQL schema S, mode m ∈ {off, lex, parse, parse+guards}, top-k cutoff k); output = (warped score vector s̃_t over V used by beam search)` - Pseudocode:

```
function PICARD_score(h, s_t, S, m, k):
    s̃_t ← [-∞ for v in V]                  # reject by default
    top ← argTopK(s_t, k)
    for v in top:
        text ← detokenize(h · v)
        if m == off:
            ok ← True
        else if m == lex:
            ok ← lex_ok(text, S)             # keywords/identifiers valid
        else if m == parse:
            ok ← parse_ok(text, S)           # partial AST + tid.cid resolves
        else:  # parse+guards
            ok ← parse_ok(text, S) and guards_ok(text, S)
                 # alias/table must be (or will be) in FROM scope;
                 # column cid must be reachable from some bound table
        if ok:
            s̃_t[v] ← s_t[v]
    return s̃_t

# Beam search calls PICARD_score at every step; rejected hypotheses
# drop out of the beam immediately, never occupying a slot.
```

Workflow.

```
fine-tune unmodified T5 (Base, Large, or 3B) on Spider/CoSQL with schema in input
   │
   ▼
beam search of width b (paper uses 4)
   │
   ▼
for each live hypothesis h:
   query LM head -> s_t over V
   take top-k tokens (k ∈ {2,4,8})
   for each candidate v:
     append, detokenize -> partial string
     hand to attoparsec parser configured in mode m
     accept/reject -> score = s_t[v] or -∞
   recombine, prune to beam width b
   │
   ▼ (repeat until EOS)
finalize highest-scoring complete hypothesis
```

Dataset format. `(natural language question, serialized DB schema [+ optional cell values], gold SQL query)`; for CoSQL, prepend prior interaction turns in reverse chronological order, truncated to 512 T5 tokens.

Result. With T5-3B + PICARD (parse-with-guards, $k = 2$, beam $= 4$), exact-set-match on Spider rises from 69.9 to 75.5 on dev and 68.0 to 71.9 on test, and execution accuracy reaches 79.3 dev / 75.1 test, beating the LGESQL+ELECTRA baseline (75.1 dev / 72.0 test EM). On CoSQL, T5-3B+PICARD lifts question-match from 51.4 to 54.6 and interaction-match from 21.7 to 23.7 on test. Decoding cost rises modestly from 2.5 s/sample to 3.1 s/sample on an A100. Invalid-SQL rate for T5-3B drops from 12% to 2%.

Benchmark	Model	Metric	Their result	Best prior baseline
Spider (test)	T5-3B + PICARD	EM% / EX%	71.9 / 75.1	LGESQL+ELECTRA 72.0 / —

SoundCode delta. - **Same as SoundCode:** External, language-aware verifier wrapped around an unmodified pre-trained LM at inference; verification operates on the *detokenized surface form* rather than internal model state; the verifier is mode-tiered (lex, parse, parse+guards) much like SoundCode's syntactic-vs-semantic check tiers (`cargo check` vs LSP diagnostics); rejections cause beam re-routing analogous to SoundCode's rollback to a verified state. - **Different:** PICARD is fully *synchronous* — every single token is parsed before being committed — and works in beam search (re-rank, no rollback because rejected tokens never enter a hypothesis). SoundCode runs `cargo check` and rust-analyzer LSP *asynchronously* only at depth-0 structural boundaries (`;`, `}`), tolerates speculative tokens between checkpoints, and explicitly *rolls back* on blocking diagnostics. PICARD's parser is hand-written for SQL with schema-aware guards; SoundCode reuses existing off-the-shelf Rust tooling. PICARD targets a small fixed grammar (SQL) on T5 fine-tunes; SoundCode targets a Turing-complete language on general code LLMs without fine-tuning. - **Implication for SoundCode design:** PICARD shows that even cheap per-step structural rejection delivers large accuracy gains and can lift a small model to match a much larger unconstrained one — motivating SoundCode's bet that LSP-guided pruning lets a small Rust model approximate a larger one. But PICARD also demonstrates the cost ceiling of synchronous per-token checking (3.1 s/sample for SQL); for Rust, where `cargo check` is orders of magnitude slower than monadic SQL parsing, SoundCode must (i) batch checks at structural boundaries, (ii) run them asynchronously off the decode loop, and (iii) restrict expensive checks to a small `k` of structurally significant rollback points rather than every token. PICARD's "guards" idea — eager semantic side-conditions that fail-fast before the AST closes — maps directly to SoundCode's choice to surface LSP diagnostics (unresolved names, type mismatches) at the first `;`/`}` rather than waiting for whole-file compilation.

Synchromesh: Reliable Code Generation from Pre-trained Language Models (Poesia et al., 2022) TL;DR. Synchromesh wraps a frozen LLM with two add-ons to make code generation reliable: Target Similarity Tuning (TST), which fine-tunes a sentence embedder to retrieve few-shot examples whose *target programs* (not surface utterances) resemble the desired output, and Constrained Semantic Decoding (CSD), a per-token decoding-time filter that uses a hand-written *Completion Engine* (CE) per DSL to mask the LLM's next-token logits so only tokens extending to a valid program are sampled. Token-vocabulary vs. DSL-token misalignment is reconciled via Brzozowski derivatives of regular expressions returned by the CE. Evaluated on SQL (Spider), Vega-Lite (NLV), and SMCalFlow with GPT-3 13B/175B and Codex 175B; CSD+TST raises Codex SQL execution accuracy from 56% to 64% and Vega-Lite validity from 87% to 99%.

Notations.

Symbol	Meaning
Σ	Base alphabet (characters) of the target language
Σ_L^*	Strings of target-language tokens; $L \subseteq \Sigma_L^*$ is the set of valid programs
L^c	Prefix-closure of L: partial programs extendable to some $p \in L$

Symbol	Meaning
$C_L : \Sigma_L^* \to 2^\Sigma$	Completion engine: partial function from a partial program to a regex over valid next characters
Σ_M	LLM's BPE vocabulary (distinct from Σ_L)
$V_M(s)$	Set of LLM tokens that keep s in L^c
$u^{-1}S$	Brzozowski derivative of language S w.r.t. string u: $\{v : uv \in S\}$
$S(p_a, p_b) \in [0, 1]$	Normalized program-similarity (tree edit distance over ASTs) used as TST target

Definitions. - **Conceptual error:** generated code misses user intent (wrong column, wrong API). - **Implementation error:** code matches intent in structure but fails syntax/scope/type-check/execution. - **Completion point:** a partial program p at which $C_L(p)$ is defined (i.e., a maximal-match boundary). - **Completion Engine (CE):** axiomatized partial function C_L s.t. (A1) ϵ and every $p \in L$ are completion points and $C_L(p) = r'\$'$ for a regex r'; (A2) if $C_L(s)$ matches t then st is also a completion point; (A3) exhaustiveness — if $s = tt_0$ extends a completion point by token t_0, then t is a completion point with $t_0 \in C_L(t)$. - **Two-layer CE:** context-free layer auto-derived from an ANTLR LL(*) grammar (Augmented Transition Network); context-sensitive layer encodes scope/schema/types per language. - **Target Similarity Tuning (TST):** fine-tunes Sentence-BERT f_θ so utterance embeddings predict program-level AST similarity, used for nearest-neighbour few-shot retrieval.

Equations.

$$\mathcal{L}_{TST}(\theta) := \mathbb{E}_{i,j\sim\mathcal{D}}\,[\,f_\theta(u_i, u_j) - S(p_i, p_j)\,]^2 \quad (1)$$

Mean-squared regression of the sentence-embedding similarity onto AST similarity over pairs $(u_i, p_i), (u_j, p_j)$ in training bank $\mathcal{D}$.

$$V_M(s) = \{\, t \in \Sigma_M : s{\cdot}t \in L^c \,\} \quad (2)$$

Valid next-token set at LLM step s: BPE tokens whose concatenation keeps the prefix completable.

$$u^{-1}S = \{\, v : uv \in S \,\} \quad (3)$$

Brzozowski derivative; used to test whether the remainder of s past the last completion point can be extended into $C_L(p)$ in linear time.

Algorithm. - I/O tuple: `input = (LLM M, M's vocab Σ_M, completion engine C_L); output = (string s ∈ L sampled token-by-token with all CE constraints satisfied)` - Pseudocode (CSD + ValidPrefix decision procedure for L^c):

```
CSD(M, Σ_M):
  s ← ""; next_token ← ""
  while next_token != "$":                          # "$" = stop
    valid_tokens ← {t ∈ Σ_M | ValidPrefix(s · t)}    # mask vocab
    next_token  ← Sample(M(s), valid_tokens)         # logit-bias
    s           ← s · next_token
  return s
```

```
ValidPrefix(s):
  p ← ""; next_prefix ← ""
  while next_prefix != ⊥:                              # peel completion points off s
    p           ← p · next_prefix
    s           ← next_prefix⁻¹ · s
    regex       ← C_L(p)                                 # CE call
    next_prefix ← startswith(s, regex)                  # longest prefix matching regex
  return (s⁻¹ · regex ≠ ∅)                            # Brzozowski-derivative emptiness test
```

Optimizations: order tokens by length and rejected-prefix Trie pruning (BPE); memoize completion points; for black-box APIs (OpenAI), rejection sampling — generate a full program, scan token-by-token, splice in ≤ 15 single-token CSD corrections.

Workflow. Text flowchart: 1. User utterance u enters Synchromesh. 2. S-BERT + TST embedder retrieves k=5 nearest training examples by *program* similarity. 3. Prompt = few-shot examples + u, sent to LLM (GPT-3/Codex via OpenAI API). 4. CSD decoding loop starts with $s = \epsilon$; ANTLR-derived context-free layer + hand-written context-sensitive layer of CE expose $C_L(p)$ regex at each completion point. 5. At each LLM step, compute $V_M(s)$ by running `ValidPrefix` on $s \cdot t$ for candidate t; emptiness of Brzozowski derivative answers membership in L^c. 6. Apply logit bias so only $V_M(s)$ is sampleable; draw next token; append to s. 7. Repeat until end-of-program token "$" emitted; resulting s is in L by construction. 8. No rollback — invalid extensions are rejected *before* commitment, not undone after.

Dataset format. `(utterance u, program p)` pairs in a per-DSL bank $\mathcal{D}_i = (p_i, u_i)$: Spider (SQL, train/val split), NLV Corpus (Vega-Lite, leave-one-dataset-out across 3 datasets), SMCalFlow (LISP-like calendar DSL).

Result. SQL execution accuracy, Vega-Lite/SMCalFlow exact-match accuracy. CSD+TST adds 8–25 absolute points over base LLMs; validity often reaches 97–99%. CSD overhead averages 8% wall-clock with direct logit access; up to 15 corrections under API access. Still trails fully supervised systems by 9–11 pts post-augmentation.

Benchmark	Model	Metric	Their result	Best prior baseline
Spider (SQL)	Codex 175B + CSD + TST	Execution accuracy	64%	56% (Codex alone) / 79% (Scholak et al. supervised)

SoundCode delta. - **Same as SoundCode:** wraps a frozen LLM with an external language-aware verifier and *intervenes during decoding* (not after); rejects bad continuations rather than repairing post-hoc; treats validity as a first-class metric distinct from accuracy. - **Different:** (i) verification granularity is *per BPE token* via a hand-coded CE + Brzozowski derivatives — no parser/compiler in the loop — whereas SoundCode invokes `cargo check` + rust-analyzer at structural boundaries (depth-0 `;`, `}`); (ii) Synchromesh is *prevention by construction* (mask the logits), with no rollback; SoundCode commits speculative tokens and performs *structural rollback to the last verified checkpoint* on a blocking diagnostic; (iii) Synchromesh is *synchronous and blocking* — CSD must finish before each token; SoundCode runs verification *asynchronously* alongside token streaming; (iv) targets DSLs with bespoke per-language CEs (SQL, Vega-Lite, SMCalFlow); SoundCode targets a Turing-complete language (Rust) and explicitly notes Synchromesh's authors deferred this as future work. - **Implication for SoundCode design:** Synchromesh validates that constraint-aware decoding gives 8–25 pt accuracy gains with ~8% overhead and near-100% validity, so SoundCode's hypothesis that *some* external semantic feedback during generation pays off is well-supported. But because Synchromesh requires hand-writing a CE per language and stays purely syntactic/schematic,

scaling to Rust's borrow checker and trait resolution is impractical that way — motivating SoundCode's choice to reuse the existing `cargo`/rust-analyzer pipeline as the "completion engine" and to absorb its latency via asynchrony + rollback rather than per-token blocking masking.

Grammar-Constrained Decoding for Structured NLP Tasks without Finetuning (Geng et al., 2023) **TL;DR.** GCD intervenes inside the autoregressive decoding loop of an off-the-shelf LM by using an incremental parser as a *completion engine*: at every step, the next-token distribution is masked to the subset of vocabulary tokens that keep the partial generation a valid prefix of a string in a user-supplied context-free grammar G. The paper formalizes 14 structured NLP tasks as formal languages, introduces *input-dependent grammars* (IDGs) where G depends on the input x (e.g., the candidate-entity set in entity disambiguation), and shows that few-shot GCD with LLaMA-33B matches or beats fine-tuned task-specific baselines (GenIE, GENRE) on cIE and ED without any training. Grammar masking happens on CPU in parallel with the LM forward pass; latency overhead is negligible for small grammars and modest for the 2.7M-entity cIE grammar.

Notations.

Symbol	Meaning
$x = \langle x_0, \ldots, x_{n-1} \rangle$	Input token sequence (e.g., words of a sentence)
$y = \langle y_0, \ldots, y_{m-1} \rangle$	Output token sequence the LM must generate
$G = (V, \Sigma, P, S)$	Character-level context-free grammar: non-terminals V, terminals Σ, productions P, start symbol $S \in V$
$G_{\text{tok}} = (V, \Sigma_{\text{tok}}, P_{\text{tok}}, S)$	Token-level grammar obtained by running the LM tokenizer over Σ and P
ε	Empty string
α	Length-normalization exponent in score S/m^{α}

Definitions. - *Completion engine* (Poesia et al., 2022 terminology): a function $C(G, y_{<t})$ that, given the grammar and the partial generation, returns the set of next allowed terminals/tokens. - *Incremental parser*: an Earley-style parser (the Grammatical Framework parser of Angelov, 2009) that maintains parse state across tokens so that C runs in time proportional to the new token, not the whole prefix. - *Input-dependent grammar (IDG)*: a grammar $G(x)$ built from the input x; 13 of the 14 surveyed tasks need this — e.g., for entity disambiguation $G(x)$ enumerates only the candidate entities for mention m. - *Input-independent grammar (IIG)*: a single grammar reused for all inputs; weaker constraint, used as ablation. - *Abstract vs. concrete grammar*: GCD writes one abstract grammar G at the character level, then auto-derives a concrete G_{tok} per tokenizer (BPE, SentencePiece, ...) so the same spec works across LMs. - *Validity*: a generation y is *valid* iff it is in $L(G_{\text{tok}})$; GCD guarantees validity by construction. - *Likelihood misalignment*: pathology where the grammar-constrained arg max collapses to the empty-string sentence "$" because $p(\$) > p(\text{any valid non-empty prefix})$ under the unconstrained LM.

Equations.

$$G_{\text{tok}} = (V, \Sigma_{\text{tok}}, P_{\text{tok}}, S) \quad (1)$$

Same non-terminals/start symbol as G; terminals are LM tokens; rules P_{tok} obtained by tokenizing the strings in P.

$$p_{\text{GCD}}(y_t \mid y_{<t}, x) \;\propto\; p_{\text{LM}}(y_t \mid y_{<t}, x) \cdot \mathbf{1}[y_t \in C(G(x), y_{<t})] \quad (2)$$

Take the LM logits and zero out every token not in the allowed set C, then renormalize.

$$\text{score}(y) = \frac{\log p_{\text{LM}}(y \mid x)}{m^{\alpha}} \quad (3)$$

Length-normalized beam score; with $\alpha \geq 2$, the empty-string $ stops winning the beam.

Algorithm. - I/O tuple: `input = (LM p_LM, abstract grammar G, input x, tokenizer T, beam size k, length-norm α); output = (valid token sequence y ∈ L(G_tok))` - Pseudocode:

```
1: G_tok ← compile(G, T)                # one-time tokenizer adaptation
2: parser ← IncrementalParser(G_tok)    # GF / Earley
3: beams ← [(⟨⟩, parser.initial_state(), 0.0)]
4: for t = 0, 1, 2, …:
5:     new_beams ← []
6:     for (y_<t, state, score) in beams:
7:         allowed ← parser.next_tokens(state)        # completion engine C
8:         logits ← p_LM(· | x, y_<t)                  # GPU forward pass
9:         logits[v ∉ allowed] ← −∞                    # grammar mask
10:        for v in top-k(softmax(logits)):
11:            state' ← parser.advance(state, v)
12:            new_beams.append((y_<t · v, state', score + log p(v)))
13:    beams ← top-k by length-normalized score (eq. 3)
14:    if all beams ended in EOS: break
15: return argmax_y score(y) / m^α                    # skip empty "$"
```

Workflow. Text flowchart:

```
user writes abstract CFG G (BNF / GF)
    │
    ▼
compile G --tokenizer-adapt--> token-level G_tok          [one-time, CPU]
    │
    ▼
for each input x:
    build G(x) if grammar is input-dependent (IDG)        [CPU]
    │
    ▼
decode step t:  LM forward(x, y_<t) ──► logits  [GPU]
                parser.next_tokens(y_<t) ──► allowed set [CPU, in parallel]
                logits ⊙ mask(allowed) ──► constrained dist
                sample/beam-search ──► y_t, advance parser state
    │ (repeat until EOS or max length)
    ▼
return top-1 non-empty beam (length-normalized)
```

Dataset format. `(x, y_gold)` where x is a natural-language input sequence and y_{gold} is a *linearized* string of the structured target — e.g., cIE: `$y = $ "[s] Witchita [r] cast member [o] John Smith [s] Witchita [r] instance of [o] film"`; CP: bracketed Penn-Treebank string; ED: mention followed by `[Canonical entity]`.

Result. GCD with LLaMA-33B in the 4-shot setting matches the supervised state-of-the-art on cIE (beats GenIE-T5-base, the bespoke supervised model) and on ED beats GENRE-AIDA-only. On constituency parsing it boosts LLaMA but stays well below supervised parsers; crucially, IDG raises parse-tree *validity* from 64% (unconstrained) to 100%.

Benchmark	Model	Metric	Their result	Best prior baseline
cIE / SynthIE-text-small (4-shot)	LLaMA-33B + GCD	micro-F1	**36.0**	34.8 (GenIE T5-base, supervised)
ED / 6-dataset avg. (4-shot, IDG)	LLaMA-33B + GCD	micro-acc	80.3	89.4 (ReFinED, supervised)
CP / PTB ≤64 tok (8-shot, IDG)	LLaMA-33B + GCD	F1 / Validity	54.6 / **100.0**	95.7 / 100.0 (Zhang et al. 2020)

SoundCode delta. - **Same as SoundCode:** Intervenes inside the autoregressive decoding loop of an off-the-shelf LM to enforce a *structural* correctness property (here: membership in a CFG; in SoundCode: type/borrow-correctness via `cargo check`+LSP). Both treat constraint-checking as a CPU-side companion to a GPU forward pass and aim for "no fine-tuning, swap-in across models." Both report a constraint-engine latency table separate from LM latency. - **Different:** GCD is **per-token, synchronous, hard-mask**: at every step the parser must enumerate the next-allowed token set before the LM may emit, and any disallowed token is impossible (probability zero). There is **no rollback** — the parser only ever extends a guaranteed-valid prefix. The constraint is **purely syntactic** (CFG/PMCFG); it cannot express name resolution, type-checking, or borrow rules. Grammars are **input-dependent only in the trivial sense** of enumerating known candidate strings (entities, words); they cannot react to *generated* program state. SoundCode is the opposite design point: **asynchronous, structural-boundary checkpoint**, **soft** (the LM may emit any token in free flight), with **rollback** to the last verified `;/}` on a blocking diagnostic, and the verifier is a *semantic* tool chain (rustc + rust-analyzer) that knows types and lifetimes. - **Implication for SoundCode design:** GCD sets the upper bound on what a *fully* synchronous, syntactic constraint can do — bracket-balance and vocabulary coverage are essentially free (1-4 ms/token), so SoundCode should not bother re-implementing those guarantees with the LSP; let an EBNF for the Rust surface syntax handle them cheaply, and reserve the expensive asynchronous LSP/`cargo check` tier for semantic checks (unresolved names, type mismatches, borrow errors) that no CFG can express. GCD's "likelihood misalignment" pathology (the model prefers the empty completion under hard constraints) is a warning for SoundCode: when rollback discards a hypothesis, the resampled distribution may also collapse to a degenerate continuation, so SoundCode needs an analogue of length normalization (e.g., a per-checkpoint prompt-history guard) to avoid oscillation.

DOMINO: Guiding LLMs The Right Way: Fast, Non-Invasive Constrained Generation (Beurer-Kellner et al., 2024) TL;DR. DOMINO is a context-free-grammar (CFG) constrained-decoding algorithm that aligns sub-word LLM tokens with grammar terminals via pre-computed per-state "subterminal" prefix trees, so the next-token mask can be obtained by a tiny tree traversal instead of scanning the full vocabulary at every step. Two further optimizations — opportunistic masking (verify the LLM's argmax against the parser first, only compute the rest of the mask on rejection) and a count-based speculative decoder over (scanner-state, parser-state) pairs — push wall-clock overhead to near-zero or even net speedup. On GSM8K and CoNLL2003 with Mistral-7B / Llama-2-13B, DOMINO matches or

exceeds unconstrained accuracy (where every other method loses up to 11 points) while running up to 2.71x faster than unconstrained generation.

Notations.

Symbol	Meaning
$\mathcal{V}$	LLM sub-word vocabulary (set of tokens)
$G,\ L_G$	Context-free grammar and the language it describes
$r_i,\ L_R$	Regex of the i-th terminal of G; language of $R = r+$ over the chained terminal NFAs, with $L_G \subseteq L_R$
$S,\ P$	Character-level scanner (NFA) and online parser tracking grammar state
α	Currently-read terminal (or sub-terminal) of G
T_q	Per-scanner-state prefix tree mapping vocabulary tokens to the subterminal sequences they yield
k	Lookahead depth into T_q when computing the next-token mask
$m,\ v,\ v'$	Boolean mask over $\mathcal{V}$; raw logits; masked logits $v' = m \odot v$
s	Number of speculative tokens emitted per step

Definitions. A constrained decoder is *minimally invasive* (Def. 2.1) iff every output that an unconstrained model could produce for a prompt is also reachable under the constrained model on the same prompt — i.e., the constraint only removes illegal outputs, never reshapes the conditional. The paper distinguishes *full*, *start*, *end*, and *continuation* sub-terminals of a terminal NFA depending on whether a token's character sequence traverses through accepting/non-accepting states $q_0 \to q_\alpha$. A *bridge token* is a single vocabulary token whose characters straddle two grammar terminals (e.g., `}\n` spans `}` and whitespace); naive masks drop these and force higher-perplexity, badly-tokenized continuations. *Opportunistic masking* sidesteps mask computation by first asking the parser to validate the LLM's argmax. *Retokenization* (App. B) is the procedure used to compute the model-preferred tokenization of a fixed string for the perplexity comparison.

Equations. The only numbered formula is the speculative next-token prior conditioned on the joint state (α, β) of scanner sub-state α and parser sub-state β:

$$\mathbb{P}(l \mid \alpha, \beta) = \frac{\#\{\text{LLM chose } l \text{ in state } (\alpha, \beta)\}}{\#\{\text{reached state } (\alpha, \beta)\}}$$

Reading: a simple count-based estimator predicts the next vocabulary token from the joint scanner/parser state; whenever this prior places near-all mass on one token, that token can be emitted speculatively, skipping an LLM forward pass. The masked-logit step $v' = m \odot v$ from Algorithm 1 (line 7) zeros out logits of grammar-illegal tokens before arg max / sampling.

Algorithm. - I/O tuple: `input = (CFG G, alphabet Σ, vocabulary V, LLM f, tokenized prompt x, lookahead k); output = (token sequence o with detok(o) ∈ L_G)` - Pseudocode (offline pre-compute + online decode):

```
# Offline: Construct Terminal Tree (Algo 2)
build scanner S = chained NFA over r_1,...,r_n,r_EOS  # Lemma 3.1
for each scanner state q in S.states():
    alpha = q.subterminal()
    T = {}
    for each token l in V:
      for each subterminal sequence {alpha_1^j,...,alpha_{m_j}^j}_j = q.traverse(l):
            T += {(alpha_1^j,...,alpha_{m_j}^j, l)_j}
    T_q = PrefixTree(T)

# Online: Constrained Decoding (Algo 1 + DOMINO mask)
o = []; C.init()                              # parser+scanner checker
loop:
    C.update(o)                               # advance scanner S and parser P
    q = S.active_state(); P_state = P.state()
    # Opportunistic mask:
    v   = f(x + o); t_hat = argmax v
    if parser-path-from-T_q-root-to-t_hat exists: m = {t_hat}
    else: m = union of tokens reachable in T_q within depth k that P accepts
    v'  = m ⊙ v
    t   = decode(v')                          # argmax or sample
    if t == EOS: break
    o.append(t)
    # Speculative extension (optional, s tokens):
    while spec_count < s and P(l | alpha, beta) is peaked:
        o.append(argmax_l P(l | alpha, beta))
return o
```

Workflow. Text flowchart (offline then per-token):

1. From CFG G derive terminal regexes $r_1, \ldots, r_n$ and $r_{\text{EOS}} = \$$.
2. Compile chained NFA scanner S via the ε-disjunction construction (Lemma 3.1) so any prefix of L_R is recognized.
3. For each scanner state q, enumerate every $l \in \mathcal{V}$ as a sequence of sub-terminals and store them in a prefix tree T_q (Algo 2).
4. At inference: feed the prompt; in each step advance S and the online parser P on the last emitted token.
5. Run the LLM forward pass to get logits v.
6. Try opportunistic masking: validate $\arg\max v$ against T_q + parser; accept if legal.
7. Otherwise traverse T_q to depth k (use $k = \infty$ for full minimal-invasiveness) and form mask m from parser-legal leaves.
8. Decode $\arg\max(m \odot v)$ or sample; emit token.
9. Consult the count-based prior $\mathbb{P}(l \mid \alpha, \beta)$ to emit up to s speculative tokens before the next LLM call.
10. Stop on EOS terminal.

Dataset format. Tuple: $(\text{prompt } x, \text{CFG/regex/template } G, \text{reference answer } y^\star, \text{tokenizer, model } f)$. Evaluation prompts use 5-shot demonstrations from the training split; outputs are JSON instances structured by G (see App. C/D Listings 3–9 for the JSON, GSM8K-reasoning, C-program, XML, and fixed-template grammars).

Result. DOMINO is the only constrained decoder in the comparison that preserves unconstrained task accuracy across both Mistral-7B and Llama-2-13B, while *also* increasing throughput (up to 2.71x on CoNLL2003 / Llama-2-13B and 1.91x on the fixed-template grammar). Competing methods either drop accuracy by up to 11 points (GUIDANCE template) or

slow inference to ~0.74–0.86x (llama.cpp, GUIDANCE WS). Lookahead $k = \infty$ is required to fully recover accuracy: $k = 0$ collapses Llama-2 on GSM8K from 0.155 to 0.0 because bridge tokens like `},` are unreachable.

Benchmark	Model	Metric	Their result	Best prior baseline
GSM8K (JSON-constrained, 5-shot)	Mistral-7B	Accuracy / throughput vs unconstrained	0.418 / 1.77x	0.403 / 0.54x (GUIDANCE)

SoundCode delta. - **Same as SoundCode:** Both treat token-grain alignment as a first-class problem and try to minimize *invasive* intervention so the LLM's preferred token distribution survives; both inject auxiliary state (parser/scanner in DOMINO, LSP + `cargo check` in SoundCode) into an otherwise autoregressive loop, and both lean on a pre-computed structure (subterminal prefix trees vs. structural boundaries `;/}`) to make verification cheap. Both also report wall-clock overhead as a primary metric and treat low-overhead as essential to deployability. - **Different:** DOMINO is a *hard* syntactic constraint enforced *every* step via masking; SoundCode performs *semantic* (type/borrow) verification *only at depth-0 `;/}` boundaries* and rolls back asynchronously on a blocking LSP diagnostic — it cannot mask logits to prevent type errors mid-token. DOMINO never backtracks (the mask guarantees legal tokens); SoundCode's defining mechanism is structural rollback to a checkpoint. DOMINO's speculation is a count-based prior over parser state; SoundCode's "speculation" is the standard autoregressive draft that the LSP verifier later confirms or rejects. - **Implication for SoundCode design:** (1) Adopt DOMINO-style pre-computed terminal trees as a cheap front-end *sub-grammar* layer to prevent the LLM from emitting tokens that are obviously not valid Rust lexemes — this would reduce useless cargo-check invocations downstream, complementing rather than replacing the semantic LSP gate. (2) Borrow the *opportunistic-masking* idea: before asking the LSP about a checkpoint, first check a fast local property (e.g., balanced braces); only when that fails escalate to the expensive verifier. (3) DOMINO's near-zero overhead establishes a hard baseline: any SoundCode overhead above ~1x must be justified by semantic catches that pure syntactic constraint cannot make.

1.C Speculative-decoding family — structural blueprint

The three papers below established the structural skeleton SoundCode reuses: *speculate ahead, verify in parallel, accept-or-roll-back at the next step.* All three use the LLM itself as verifier — speculative decoding compares a small draft model's tokens to the large target model's distribution; Lookahead Decoding uses Jacobi iteration over the same LLM. SoundCode borrows the control-flow skeleton and swaps the LM-as-verifier for a *semantic* verifier (`cargo check` + LSP). The structural insight transfers; the verifier is what differs.

Fast Inference from Transformers via Speculative Decoding (Leviathan et al., 2023)
TL;DR. Speculative decoding accelerates autoregressive Transformer inference by running a cheap approximation model M_q to draft γ guess tokens, then verifying all γ guesses in a single parallel call to the expensive target model M_p via a novel *speculative sampling* rule that preserves M_p's exact output distribution. Accepted guesses commit, the first rejected guess is replaced by a draw from a residual distribution $\text{norm}(\max(0, p - q))$, and the next iteration resumes from the new prefix. On T5-XXL (11B) translation and summarization, the method delivers 2.6×–3.4× wall-clock speedups with identical outputs and no retraining.

Notations.

Symbol	Meaning
M_p	Target (large, slow) autoregressive model being accelerated
M_q	Approximation (small, fast) draft model
$p(x), q(x)$	Next-token distributions $p(x_t \mid x_{<t})$ from M_p, M_q for a fixed prefix
$\gamma \in \mathbb{Z}^+$	Number of draft tokens proposed per iteration
$\beta_{x_{<t}}$	Acceptance probability of one guess: $\Pr_{x\sim q}[\text{accept } x]$
$\alpha = E(\beta)$	Expected acceptance rate (i.i.d. assumption)
c	Cost coefficient: ratio of one M_q call to one M_p call

Definitions. A *guess* is a token sampled autoregressively from M_q. An *iteration* of the algorithm proposes γ guesses, runs M_p on all γ extended prefixes in parallel, then accepts a prefix-maximal run of guesses. *Speculative sampling* is the rule: accept guess $x \sim q(x)$ unconditionally if $q(x) \leq p(x)$, else accept with probability $p(x)/q(x)$; if rejected, draw a replacement from $p'(x) = \text{norm}(\max(0, p(x) - q(x)))$. The *acceptance rate* $\beta_{x_{<t}}$ is the probability one such guess is accepted given prefix $x_{<t}$. The *natural divergence* $D_{LK}(p, q) = \sum_x |p(x) - M(x)|$ with $M(x) = (p(x) + q(x))/2$ summarizes model disagreement; Corollary 3.6 shows $\alpha = 1 - E(D_{LK}(p, q))$.

Equations.

$$E(\#\text{generated tokens}) = \frac{1 - \alpha^{\gamma+1}}{1 - \alpha} \quad (1)$$

Under i.i.d. guesses with success probability α, the number of tokens produced per iteration is a capped geometric variable with cap $\gamma + 1$; this is the expected tokens per single run of M_p.

$$\text{Expected walltime improvement} = \frac{1 - \alpha^{\gamma+1}}{(1 - \alpha)(\gamma c + 1)} \quad (\text{Thm } 3.8)$$

Speedup over baseline as a function of model agreement α, draft length γ, and relative draft cost c; numerator is expected tokens per iteration, denominator is per-iteration cost in units of one M_p call.

$$\text{Expected ops increase} = \frac{(1 - \alpha)(\gamma \hat{c} + 1)}{1 - \alpha^{\gamma+1}} \quad (\text{Thm } 3.11)$$

Total arithmetic operations grow by this factor (with $\hat{c}$ the per-token compute ratio); concurrency is "free" only if hardware supports $\gamma + 1$ parallel M_p evaluations.

Algorithm. - I/O tuple: `input = (M_p, M_q, prefix, gamma); output = prefix' (extended by 1 to gamma+1 tokens, distributed identically to M_p alone)` - Pseudocode (Algorithm 1, SpeculativeDecodingStep):

```
Inputs: M_p, M_q, prefix
# 1. Draft gamma guesses autoregressively from M_q
for i = 1..gamma:
    q_i(x) <- M_q(prefix + [x_1, ..., x_{i-1}])
    x_i    ~ q_i(x)
# 2. Verify in parallel with one batched M_p call
p_1(x), ..., p_{gamma+1}(x) <-
    M_p(prefix), M_p(prefix+[x_1]), ..., M_p(prefix+[x_1..x_gamma])
# 3. Decide acceptances via speculative sampling
draw r_1,...,r_gamma ~ U(0,1)
n <- min({ i-1 : r_i > p_i(x_i)/q_i(x_i) } u {gamma})
# 4. Resample the first rejected token (or sample bonus token)
p'(x) <- p_{n+1}(x)
if n < gamma:
    p'(x) <- norm(max(0, p_{n+1}(x) - q_{n+1}(x)))
t ~ p'(x)
return prefix + [x_1, ..., x_n, t]
```

Workflow.

```
prefix --> M_q draft loop --> gamma guesses (x_1..x_gamma)
                                        |
                                        v
            single parallel M_p call on gamma+1 prefixes
                                        |
                                        v
       sequential speculative-sampling check per guess
        (accept iff r_i <= p_i(x_i)/q_i(x_i), else reject)
                                        |
              n = index of first rejection (or gamma)
                                        |
                +-----------------------+------------------------+
                | n = gamma (all accepted)         | n < gamma (rejection at n+1)
            | sample bonus t ~ p_{n+1}       | sample replacement t ~ norm(max(0,p-q))
                +-----------------------+------------------------+
                                        v
                new prefix = prefix + [x_1..x_n, t]; loop
```

Dataset format. Tuple (`prefix_tokens`, `target_model` `M_p`, `draft_model` `M_q`, `gamma`, `sampling_temp`) → generated continuation matching M_p's distribution exactly; benchmarks supply the prefix (WMT EnDe source sentences, CCN/DM articles, lm1b, LaMDA dialog turns).

Result. T5-XXL (11B) with T5-small (77M) draft yields the best trade-off. Speculative decoding achieves 2×–3× wall-clock speedups on translation and summarization with identical outputs and no retraining. Empirical α rises with draft model size and is higher under argmax than stochastic sampling. Even trivial bigram drafts give a 1.25× speedup on EnDe.

Benchmark	Model	Metric	Their result	Best prior baseline
WMT EnDe (T5X argmax)	T5-XXL 11B with T5-small 77M draft, $\gamma = 7$	Wall-clock speedup	3.4× ($\alpha = 0.75$)	1.0× (standard T5X)

SoundCode delta. - **Same as SoundCode:** Producer/verifier split with parallel verification;

commit-or-rollback at boundaries; verifier's accept/reject decision governs whether drafted tokens persist; goal is wall-clock acceleration without changing the output distribution of the underlying generator. - **Different:** Verifier is the target LLM M_p checking token-level probabilities, not a semantic oracle (`cargo check` + rust-analyzer); rollback unit is the single rejected token plus a resample from $\text{norm}(\max(0, p - q))$, not a structural checkpoint at depth-0 `;` or `}`; correctness guarantee is distributional ($x \sim p$), not semantic (compiles, type-checks); verification is synchronous within a step, not asynchronous across boundaries; targets memory-bandwidth bottleneck, not LLM-as-coder unsoundness. - **Implication for SoundCode design:** Borrow the draft/verify/commit/rollback skeleton and the "rejection triggers resample from a residual" pattern, but replace (a) per-token probabilistic acceptance with per-segment semantic acceptance gated by LSP/compiler diagnostics, and (b) the per-token resample with a structural rewind to the last verified checkpoint plus a re-prompt that includes the diagnostic. The Leviathan γ knob maps to SoundCode's choice of *checkpoint granularity* (statement vs. block); the $\alpha = 1 - E(D_{LK}(p, q))$ analysis suggests an analogous "expected-tokens-per-verification" metric driven by the LLM's empirical pass-rate at each boundary class.

Accelerating Large Language Model Decoding with Speculative Sampling (Chen et al., 2023) **TL;DR.** DeepMind's Speculative Sampling (SpS) generates a short K-token draft from a small, fast auto-regressive draft model, scores it in one parallel forward pass of the large target model, then uses a novel *modified rejection sampling* scheme to accept a left-to-right prefix while provably preserving the target distribution within hardware numerics. Because parallel scoring of a K-token continuation costs about the same wall-clock time as sampling one token on memory-bandwidth-bound large transformers, each target call emits multiple tokens. Applied to Chinchilla 70B with a 4B draft, SpS yields 2–2.5× decoding speedup on XSum and HumanEval at batch size 1, sometimes exceeding the theoretical memory-bandwidth ceiling for autoregressive decoding.

Notations.

Symbol	Meaning
$q(\cdot \mid \cdot)$	Target (large) auto-regressive model distribution
$p(\cdot \mid \cdot)$	Draft (small) auto-regressive model distribution
$x_1, \ldots, x_t$	Initial prompt sequence
$\tilde{x}_1, \ldots, \tilde{x}_K$	Tokens sampled from the draft model (lookahead)
K	Draft lookahead length (number of speculated tokens)
T	Target total sequence length
$(f(x))_+$	Normalized positive-part distribution (resampling kernel on rejection)

Definitions. Auto-regressive sampling (ArS) draws one token per forward call of q, so latency is bounded below by model-size / memory-bandwidth. SpS instead alternates a *drafting* phase (sample $\tilde{x}_1, \ldots, \tilde{x}_K \sim p$) and a *scoring* phase (one parallel call of q on the prompt plus all K drafts, yielding $K+1$ logit vectors). The *modified rejection sampling* test compares $q(\tilde{x}_t \mid \cdot)/p(\tilde{x}_t \mid \cdot)$ to a uniform $r \sim U[0, 1]$: if $r < \min(1, q/p)$ accept $\tilde{x}_t$, otherwise resample from the residual $(q-p)_+$ and exit the loop. Theorem 1 proves this recovers q exactly. If all K drafts accept, a free bonus token is sampled from the already-computed $q(\cdot \mid x_1, \ldots, x_n, \tilde{x}_1, \ldots, \tilde{x}_K)$, so the loop produces 1 to $K+1$ tokens. The target model is *unchanged* — no fine-tune, no

architecture edit — so SpS composes with quantization, multi-query attention, and Megatron sharding.

Equations.

$$\text{Accept } \tilde{x}_{n+t} \text{ iff } r < \min\left(1,\ \frac{q(\tilde{x}_{n+t} \mid x_1, \ldots, x_{n+t-1})}{p(\tilde{x}_{n+t} \mid x_1, \ldots, x_{n+t-1})}\right), \quad r \sim U[0,1] \tag{1}$$

Per-token acceptance ratio; guarantees the marginal of accepted samples matches q.

$$x_{n+t} \sim \left(q(x \mid x_1, \ldots, x_{n+t-1}) - p(x \mid x_1, \ldots, x_{n+t-1})\right)_+ \tag{2}$$

Residual resampling distribution used on rejection — concentrates mass where $q > p$.

$$(f(x))_+ = \frac{\max(0,\ f(x))}{\sum_{x'} \max(0,\ f(x'))} \tag{3}$$

Normalized positive part; makes $(q-p)_+$ a valid distribution.

$$\mathbb{P}(X = x) = \min(p(x), q(x)) + \max(0,\ q(x) - p(x)) = q(x) \tag{4}$$

Theorem 1 closure: per-position marginal of the accepted/resampled token equals the target q.

Algorithm. - I/O tuple: `input = (target q, draft p, prompt x_{1..t}, lookahead K, target length T); output = (sequence x_{1..T} distributed as q)` - Pseudocode (Algorithm 2 from the paper):

```
n <- t
while n < T:
    # 1. DRAFT phase: K serial calls of small model p
    for i = 1..K:
        x_tilde_i ~ p(. | x_{1..n}, x_tilde_{1..i-1})
    # 2. SCORE phase: ONE parallel call of large model q -> K+1 logit vectors
   compute q(. | x_{1..n}), q(. | x_{1..n}, x_tilde_1), ..., q(. | x_{1..n}, x_tilde_{1..K})
    # 3. VERIFY phase: left-to-right modified rejection sampling
    for t = 1..K:
        r ~ U[0,1]
        if r < min(1, q(x_tilde_t | ctx) / p(x_tilde_t | ctx)):
            x_{n+t} <- x_tilde_t;  n <- n + 1            # accept
        else:
            x_{n+t} ~ (q(. | ctx) - p(. | ctx))_+        # residual resample
            n <- n + 1
            break                                         # exit verify loop
    # 4. BONUS token if all K drafts accepted
    if all K accepted:
        x_{n+1} ~ q(. | x_{1..n})
        n <- n + 1
```

Workflow.

```
prompt x_{1..t}
   |
   v
```

```
[DRAFT]  serial: K calls of small p  ->  tilde_x_{1..K}
   |
   v
[SCORE]  parallel: 1 call of large q on prompt+drafts -> K+1 logit vectors
   |
   v
[VERIFY] left-to-right modified rejection sampling (per Eq. 1)
   |---- all K accept ----> emit drafts + sample bonus token from q -> K+1 tokens
  |---- reject at t -----> emit drafts_{1..t-1} + resample t from (q-p)_+ -> t tokens
   |
   v
append accepted tokens; advance n; loop until n >= T
```

Dataset format. `(prompt_tokens, max_new_tokens, decoding_params)` — text prompt fed to both target and draft; SpS itself is data-agnostic, benchmarked on `(XSum_article, summary_len=128)` 1-shot and `(HumanEval_signature, completion_len=512)` 100-shot.

Result. SpS on Chinchilla 70B (4B draft, $K = 4$, batch 1, 16 TPU v4s) attains 1.92×–2.46× mean wall-clock speedup over ArS while matching benchmark scores within numerics; on HumanEval and greedy XSum it exceeds the hard memory-bandwidth ceiling for autoregressive sampling. Acceptance rate is domain-dependent (code > summarization) because code contains many "obvious" sub-sequences the draft guesses correctly; $K = 3$–4 is the sweet spot before per-loop overhead and compounding-rejection erode the gain.

Benchmark	Model	Metric	Their result	Best prior baseline
HumanEval (100-shot, nucleus)	Chinchilla 70B + 4B draft	mean token time / pass@	5.73 ms/token, 47.0% (2.46×)	ArS: 14.1 ms/token, 45.1%

SoundCode delta. - **Same as SoundCode:** Three-phase *draft → verify → rollback-on-reject* loop with a cheap proposer (draft model here, target LLM in SoundCode) and an expensive verifier (target LLM here, `cargo check` + rust-analyzer LSP in SoundCode); verifier runs concurrently with what would otherwise be the next sequential step, exploiting that the verify cost overlaps the proposer cost; rejection triggers structural rollback to the last verified prefix and resampling from a corrected distribution. - **Different:** SpS is a *probabilistic*, distribution-preserving accelerator running at every token boundary with a per-token Bernoulli accept test calibrated by logit ratios; SoundCode is a *correctness* layer running at *structural* boundaries (depth-0 `;`, `}`) with a deterministic, semantic accept/reject from `cargo check` diagnostics. SpS uses a learned model as oracle and keeps the same distribution; SoundCode uses an external compiler/LSP oracle and intentionally *biases* the distribution away from ill-typed continuations. SpS rolls back at most K tokens, SoundCode rolls back to multi-line statement-level checkpoints. - **Implication for SoundCode design:** The SpS-style acceptance rate analysis (Figure 1 middle panel: rate decays as lookahead grows because later tokens are conditioned on earlier acceptances) maps directly to SoundCode's checkpoint-spacing choice — longer between-checkpoint spans amortize LSP latency but compound the cost of any single rejection. SoundCode should track a HumanEval-style "blocking-diagnostic rate per checkpoint" and tune checkpoint granularity to balance verifier-call overhead against rollback waste, exactly as SpS tunes K in Figure 1.

Break the Sequential Dependency of LLM Inference Using Lookahead Decoding (Fu et al., 2024) TL;DR. Lookahead Decoding is an exact, parallel decoding algorithm that accelerates autoregressive LLM inference *without* an auxiliary draft model, by reformulating

decoding as a non-linear fixed-point problem solved by Jacobi iteration. Each step runs in one forward pass a *lookahead branch* (generates many disjoint n-grams in parallel from past Jacobi trajectories) plus a *verification branch* (verifies cached n-gram candidates against the base LLM), trading per-step $\log(\text{FLOPs})$ for fewer decoding steps. On LLaMA-2-Chat 7B it achieves up to 1.8× speedup on MT-Bench and up to 4× with multi-GPU Lookahead Parallelism on code completion (ClassEval).

Notations.

Symbol	Meaning
$\mathbf{x}^0 = (x_1, \dots, x_n)$	Prompt tokens given by the user
$P_M(\cdot \mid \cdot)$	Conditional next-token distribution of base LLM M
y_i	i-th generated token
W, N, G	Lookahead window width, n-gram size, max #speculations
$\mathbf{w}^i_{1:W}$	2D lookahead window at step i, W positions of future tokens
$\mathbf{C}$	n-gram pool (cache of candidates)
α	Expected per-token acceptance rate (probability draft token passes verification)
$\mathcal{S}$	Step compression ratio = #generated tokens/#decoding steps

Definitions. - *Autoregressive decoding*: generate $y_s = \arg\max P_M(y_s \mid \mathbf{x}^0, y_{1:s-1})$ token-by-token, m sequential forward passes for m tokens. - *Guess-and-verify (speculative decoding)*: a draft model produces γ candidate tokens; the base LLM verifies them in *one* parallel pass and accepts a prefix. - *Jacobi decoding*: rewrite the m autoregressive equations as a non-linear system $f(y_i, \mathbf{y}_{1:i-1}, \mathbf{x}^0) = 0$ and iterate from a random guess $\mathbf{y}^0$ to the fixed point $\mathbf{y}^m$; converges in at most m iterations. - *Lookahead branch*: a 2D window of W future positions over $N-1$ past Jacobi trajectory steps; one parallel pass updates all W positions and emits an N-gram per position. - *Verification branch*: looks up n-grams in $\mathbf{C}$ whose first token equals the last generated token, then verifies up to G candidates in parallel; only the verified prefix is appended to the output. - *Lookahead Parallelism (LP)*: replicates the full model on each GPU and shards disjoint lookahead/verification branches across GPUs, eliminating in-step communication.

Equations.

$$y_s = \arg\max P_M(y_s \mid \mathbf{x}^0, y_{1:s-1}) \quad (1)$$

Standard autoregressive step.

$$y'_{t+i} = \arg\max P_M(y_{t+i} \mid \mathbf{y}_{1:t+i-1}, \mathbf{x}^0),\ i = 0, \dots, n-1 \quad (2)$$

Parallel verification of n guesses; accept longest matching prefix.

$$f(y_i, \mathbf{y}_{1:i-1}, \mathbf{x}^0) = 0,\ i = 1, \dots, m \quad (3)$$

Jacobi fixed-point formulation of decoding.

$$E[\#\text{tokens}] = (\gamma + 1) - \sum_{i=1}^{\gamma} (1 - \alpha^i)^b \quad (5)$$

Expected accepted tokens for b parallel speculations of length γ.

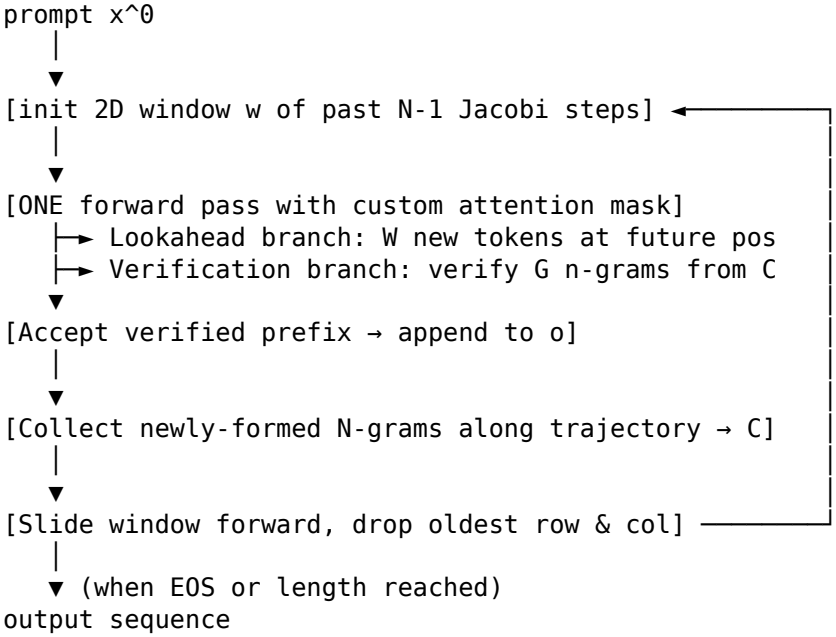
$$\mathcal{S} = (f - 1 + E[\#\text{tokens}])/f \quad (7)$$

Step compression: f steps, one of which yields $E[\#\text{tokens}]$ accepted.

Algorithm. - I/O tuple: `input = (x^0 prompt, P_M model, N n-gram size, W window size, G max speculations, m max steps); output = (o_{1:m} = (y_1, ..., y_m))` - Pseudocode (Algorithm 2):

```
C ← ∅                                    # n-gram pool
o ← ∅; o_0 ← x_n                         # accepted tokens so far
randomly init 2D window w^{2-N:0}_{1:W}
for i = 1 to m:
    # Lookahead branch: one parallel pass over W future positions
    for j = 1 to W:
        w^i_j ← argmax P_M(w^i_j | past trajectory w^{i+2-N:i-1}_j,
                                   w^{i+1-N}_{2:j}, o_{1:i}, x^0)
    # Verification branch: pull up to G n-grams from C
    g ← { n-gram from C starting with o_{i-1} }, |g| ≤ G
    o.append( VERIFY((x^0, o_{1:i}), P_M, g) )   # Alg. 3 (greedy) / 4 (sample)
    # Update pool with new N-grams from lookahead trajectory
    for j = 1 to W:
        C.add( w^{i-N+1:i}_j )
return o_{1:m}
```

Workflow.



Dataset format. `(prompt: string, reference_output: string)` for chat/summarization;

(prompt: code prefix/spec, completion: code) for HumanEval/MBPP/ClassEval; (question, answer) for GSM8K.

Result. Greedy, lossless (output distribution preserved); 1.5×–2.3× on a single A100, up to 4× with Lookahead Parallelism on 8 A100s; sampling variant via Algorithm 4 yields 1.46–1.60× on summarization.

Benchmark	Model	Metric	Their result	Best prior baseline
MT-Bench	LLaMA-2-Chat 7B	wall-clock speedup vs. HF greedy	1.88× (no FlashAttn)	1.44× (Prompt Lookup)
HumanEval (code completion)	CodeLLaMA 7B	wall-clock speedup, 1 GPU + FA	~2.65×	1.07× (HF greedy + FA)
ClassEval	CodeLLaMA-Python 13B	wall-clock speedup, 8 GPUs (LP)	~3.78×	0.78×–1.08× (TP/PP)

SoundCode delta. - **Same as SoundCode:** "Speculate now, verify in parallel, roll back rejected suffixes" structural pattern; lossless guarantee (output distribution preserved); maintains a checkpoint-like *accepted prefix* **o** and only commits verified tokens; uses a cached candidate pool (n-gram pool ↔ SoundCode's verified-checkpoint store) to bootstrap subsequent speculations. - **Different:** Verifier is the *same* LLM via Jacobi iteration (self-verification, syntactic/distributional); SoundCode's verifier is *external semantics* (cargo check + rust-analyzer LSP). Lookahead's verification is *synchronous* inside one forward pass; SoundCode's is *asynchronous* at depth-0 ;/} boundaries. Rollback unit is per-token n-gram suffix in Lookahead, vs. *structural* (last verified ;/}) in SoundCode. No notion of blocking vs. non-blocking diagnostics — Lookahead has only argmax-mismatch rejection. - **Implication for SoundCode design:** (1) The n-gram pool motif suggests caching not just the last verified checkpoint but *all* previously-verified structural fragments, enabling reuse after rollback. (2) The scaling law $\mathcal{S} \propto \log(\text{FLOPs})$ implies SoundCode's compression ceiling is bounded by verifier *latency* (LSP round-trip) rather than FLOPs, so structural granularity (how often cargo check fires) is the dominant tuning knob. (3) Lookahead's $G \propto W$ heuristic suggests SoundCode should bound concurrent in-flight LSP verifications proportionally to its speculation horizon to balance generation/verification cost.

1.D Methodological cudgel

Is Self-Repair a Silver Bullet for Code Generation? (Olausson et al., 2024) Every paper claiming "feedback improves code generation" must address this paper's matched-budget result: if a self-repair system uses k samples for repair, it must beat plain i.i.d. resampling at the same k samples. Under that bar, self-repair only wins when the feedback model is strictly stronger than the generator. SoundCode's defense is that the compiler is *strictly* stronger than the LLM on the decidable fragment it covers — types, borrow rules, name resolution — so the regime in which Olausson predicts repair beats i.i.d. *does* apply to SoundCode by construction.

TL;DR. Olausson et al. analyze LLM self-repair (generate → execute → self-feedback → repair) on HumanEval and APPS under a matched-budget protocol that charges repair for the extra samples it consumes; they find that gains over plain i.i.d. resampling are often modest, nonexistent at small budgets, and vary across data subsets. The bottleneck is the model's feedback

quality: only when the feedback model is strictly stronger than the generator (e.g., GPT-4 critiquing Code Llama) does self-repair consistently beat i.i.d., and replacing GPT-4's own feedback with that of a human programmer increases repair success by $1.58\times$ ($33.3\% \to 52.6\%$). The headline lesson is that "self-repair = i.i.d. + work" unless an external oracle stronger than the generator supplies the diagnosis.

Notations.

Symbol	Meaning
ψ	task specification (natural-language prompt + executable unit tests)
M_P	program (code-generation) model
M_F	feedback (critique) model; in self-repair $M_F = M_P$
p_i	i-th initial program sample, $p_i \overset{\text{i.i.d.}}{\sim} M_P(\psi)$
e_i	error message returned by the test bed for p_i
f_{ij}	j-th natural-language feedback string for p_i, $f_{ij} \overset{\text{i.i.d.}}{\sim} M_F(\psi; p_i; e_i)$
r_{ijk}	k-th repaired candidate, $r_{ijk} \overset{\text{i.i.d.}}{\sim} M_P(\psi; p_i; e_i; f_{ij})$
n_p, n_f, n_r	per-task budgets: # initial programs, # feedbacks per failing program, # repairs per feedback
n_{fr}	joint feedback-repair samples when $M_P = M_F$ (single forward pass yields both f_{ij} and r_{ij})
T	a repair tree rooted at ψ (Fig. 2): leaves are repair candidates
$p \vDash \psi$	program p passes all unit tests of specification ψ
k	total sample budget; for self-repair $k = n_p + n_p\, n_{fr}$

Definitions. - *Self-repair*: four-stage pipeline — code generation, code execution, feedback generation, code repair — using the same or different LLMs at each stage (Fig. 1). - *Repair tree* T: tree whose root is ψ, level 1 contains n_p initial samples, level 2 contains n_f feedback strings per failing program, and level 3 contains n_r repair candidates per feedback (Fig. 2). - *Joint sampling* ($M_P = M_F$): a single forward pass generates a (f_{ij}, r_{ij}) pair, so n_{fr} replaces (n_f, n_r) as the joint budget. - *i.i.d. baseline* (no-repair): draw k independent samples from $M_P(\psi)$ and ask whether any passes; this is the matched-budget control. - *Programs(T)*: total program samples in tree T; equals $n_p + n_p n_{fr}$ (joint) or $n_p + n_p n_f n_r$ (separate). - *Matched-budget comparison*: pass rate of self-repair at budget k is compared against pass rate of i.i.d. baseline at the *same* k — repair must "pay" for its feedback+repair samples. - *Repair success rate*: #passing repairs/#total repair candidates sampled — measures local repair efficacy, ignoring matched-budget cost.

Equations.

(1) Initial sampling: $\{p_i\}_{i=1}^{n_p} \overset{\text{i.i.d.}}{\sim} M_P(\psi)$ — draw n_p candidate programs from the generator.

(2) Feedback generation: $\{f_{ij}\}_{j=1}^{n_f} \overset{\text{i.i.d.}}{\sim} M_F(\psi; p_i; e_i)$ — sample n_f explanations of *why* p_i

failed, conditioned on the spec, program, and error.

(3) Repair: $\{r_{ijk}\}_{k=1}^{n_r} \overset{\text{i.i.d.}}{\sim} M_P(\psi; p_i; e_i; f_{ij})$ — sample n_r patched programs per feedback string.

(4) Joint feedback+repair ($M_P = M_F$): $\{(f_{ij}, r_{ij})\}_{j=1}^{n_{fr}} \overset{\text{i.i.d.}}{\sim} M_P(\psi; p_i; e_i)$ — single sample yields both critique and patch.

(5) Matched-budget pass@k: $\text{pass@}k = \Pr_{T\sim M}[\exists \text{leaf } \ell \in T : \ell \vDash \psi]$ with $k = |\text{programs}(T)| = n_p + n_p\, n_{fr}$ — probability that *any* program in the repair tree passes, charged against the i.i.d. budget k.

(6) pass@t alternative (Appx. A): $\text{pass@t} \triangleq \mathbb{E}_{\psi_d, T_d}[T_d \vDash \psi_d]$ at $t = \mathbb{E}[\text{num\_tokens}(T_d)]$ — same pass-rate definition but cost measured in tokens, accounting for verbose feedback.

Algorithm. (Experimental protocol)

- I/O tuple: `input = (task specification ψ with unit tests, models M_P and M_F, budget grid (n_p, n_f, n_r)); output = (mean pass rate vs. matched-budget i.i.d. baseline at each (n_p, n_{fr}))`

```
# Bootstrap-style estimation protocol (Sec. 3.2, Algorithm 1)
For each task ψ in the benchmark (300 APPS tasks, 164 HumanEval tasks):
  1. Sample a frozen "mega" repair tree once:
       N_p = 50 initial programs from M_P(ψ)
       For each failing p_i: N_f = 25 feedback strings from M_F(ψ; p_i; e_i)
                             N_r = 1 repair per (p_i, f_ij)   (joint when M_P=M_F)
  2. For each (n_p, n_{fr}) in {1,2,5,10,25} × {1,3,5,10}:
       Repeat N_t = 1000 times:
      Sub-sample n_p initial programs and n_{fr} feedback-repair pairs each (with replacement)
         success := any leaf passes all unit tests of ψ
       pass_rate(n_p, n_{fr}) := mean over the 1000 sub-samples
  3. Compute matched-budget i.i.d. baseline pass@k for k = n_p + n_p·n_{fr}
     by sub-sampling k programs from a separate baseline tree of size 50
  4. Heat-map cell = pass_rate(self-repair) / pass_rate(i.i.d.)   (>1 ⇒ repair wins)
# Decoding temperature = 0.8 throughout; one-shot templated prompts (Appx. G)
# Repeat with M_F ∈ {M_P, stronger model, human programmer}
```

Workflow.

```
spec ψ + unit tests
        |
        v
[M_P generates n_p initial programs] -- any pass? --> DONE (no repair needed)
        | (if all fail)
        v
[execute against tests; collect e_i error messages]
        |
        v
[M_F samples n_f feedback strings per failing program]
        |
        v
[M_P samples n_r repairs per (p_i, f_ij)]   (joint sampling when M_P=M_F)
        |
        v
[execute all repairs; success iff any leaf passes]
        |
```

v
[compare to i.i.d. baseline at k = n_p + n_p·n_{fr} samples]

Dataset format. Tuple $(\psi, \mathcal{T})$ where ψ is the natural-language specification and $\mathcal{T} = \{(x_i, y_i)\}$ is the *full* suite of executable input-output unit tests; no distinction is made between public and private tests (Sec. 2). Benchmarks: HumanEval (164 hand-written Python problems, function-completion style) and APPS-300 (random sample of 180 interview, 60 competition, 60 introductory Python problems from APPS). Models: CodeLlama-13b-instruct (open), GPT-3.5 (`gpt-3.5-turbo-0301`), GPT-4 (`gpt-4-0314`).

Result. Under matched-budget pass@k, self-repair never decisively beats i.i.d. resampling when $M_P = M_F$: on APPS-300 the largest normalized gains are $1.04\times$ for GPT-3.5 and $1.08\times$ for GPT-4 (Fig. 3, both at $n_p \geq 10, n_{fr} = 1$); on HumanEval it tops out at $1.09\times$ for CodeLlama and is $\leq 1.03\times$ for GPT-3.5 (Fig. 4); many cells are < 1 (repair *worse* than i.i.d.). Spending budget on diverse *initial* programs ($n_p \uparrow$) dominates spending it on more repairs per failure ($n_{fr} \uparrow$). The bottleneck is feedback quality: pairing weak Code Llama as M_P with GPT-4 as M_F on APPS yields large gains over both Code Llama's i.i.d. baseline and its self-repair (Fig. 5b, Sec. 4.2), and human-written feedback boosts GPT-4 repair success from 33.3% to 52.6% (Table 1, $1.58\times$). Repair success rates rise monotonically with feedback-model strength (Table 2, e.g., APPS-overall: Code Llama self-repair 1.1%, GPT-3.5 4.7%, GPT-4 10.8%).

Benchmark	Generator / Feedback model	Metric	Self-repair result	Matched-budget i.i.d.
APPS-300	GPT-4 / GPT-4 (self-repair)	normalized pass@k, $n_p = 25$, $n_{fr} = 1$	$1.08\times$ baseline	1.00 (reference)
APPS-300	Code Llama / GPT-4 (boosted feedback)	mean pass rate vs. Code Llama i.i.d.	up to $\sim 10\%$ absolute gain (Fig. 5b green)	Code Llama self-repair $\approx$ i.i.d.
APPS (40-task human study)	GPT-4 / human programmer	repair success rate (Table 1)	52.6%	GPT-4 self-feedback: 33.3%

SoundCode delta. - **Same as SoundCode:** Both consume a verifier signal (unit-test pass/fail vs. `cargo check` + rust-analyzer LSP), both treat the verifier output as conditioning for revision, and both demand matched-budget comparisons — Olausson's pass@k charges feedback+repair samples against the i.i.d. budget; SoundCode must report wall-clock and token-matched arms so any reported win is not an artifact of extra compute. - **Different:** (a) *Verifier strength*: Olausson's "feedback" is another LLM forward pass (or a human); SoundCode's verifier is a sound static analyzer that is *provably* stronger than the LLM on type/borrow-checking — type errors are decided, not opined. (b) *Granularity*: Olausson repairs whole programs after a full sample; SoundCode rolls back to the last verified structural checkpoint mid-stream, so the "repair budget" is at most a partial-suffix re-decode rather than a full re-sample plus feedback. (c) *Sampling cost*: Olausson's n_{fr} samples must each pay a full LLM forward pass for feedback; SoundCode's async LSP verification overlaps with decoding, so the dominant cost is rollback frequency, not extra LLM calls. (d) *Failure mode*: Olausson never reports cases where the model's spurious feedback poisons repair; SoundCode's analogous risk is *false-positive blocking diagnostics* (e.g., classifier demotion) which is precisely what the Week-4 evaluation found neutralized the LSP tier. - **Implication for SoundCode design:** The paper's central claim — "self-repair beats i.i.d. iff feedback model > generator" — is exactly the bar SoundCode must clear. The defensive narrative is: the Rust compiler and rust-analyzer LSP are *strictly* stronger than the LLM on the decidable fragment (type errors,

borrow errors, name resolution) they cover, so Olausson's stronger-feedback regime applies *by construction*. To honor this, SoundCode evaluations must (i) include a matched-token i.i.d. resampling baseline that pays the same wall-clock cost as rollback, (ii) report per-difficulty breakdowns (Olausson's APPS-competition shows the biggest gains — analogously SoundCode should look hardest at programs where rustc errors are dense), (iii) measure repair success rate independently of the budget so the verifier-strength claim is auditable, and (iv) ablate the verifier (`cargo check` only vs. `cargo check` + LSP) the way Olausson ablates M_F to isolate which component carries the gains. The Week-4 finding that the LSP tier blocked 0/926 calls is the SoundCode-internal version of Olausson's null result: when the "stronger" feedback is silenced in practice, repair collapses back to i.i.d.

---

2. Iterative-repair family — fast cards

The fifteen papers below address the same problem SoundCode addresses — LLMs produce wrong code on the first try — but apply the verifier *off* the critical path of decoding. They generate, then check (via tests, exec traces, compiler errors, or another LLM), then revise. Per Olausson's matched-budget result (Section 1.D), only papers whose verifier is strictly stronger than the generator escape the "self-repair = i.i.d. + work" verdict; the $M_F > M_P$**?** column in the table below tracks this.

2.0 Iterative-repair comparison table

Paper	Feedback signal	Granularity	# rounds	Headline	$M_F > M_P$?
Self-Refine (Madaan 2023)	LLM self-critique (NL)	whole solution	iterate to stop	~13% absolute on code gen vs single-shot	**no** (self)
Self-Debug (Chen 2024)	execution trace + self-explain	whole solution	iterate	TransCoder +12% (gpt-3.5)	**no** (self)
Reflexion (Shinn 2023)	pass/fail + verbal reflection	whole trajectory	episodic memory	HumanEval 91% pass@1 (GPT-4)	n/a (env signal)
CodeT (Chen 2022)	LLM-generated tests + RANSAC	parallel rerank	1 (no iter)	HumanEval pass@1 65.8% (cd-002)	**no** (self-tests)
CodeRL (Le 2022)	unit tests as RL reward	training time	n/a (offline)	APPS SOTA (CodeT5)	n/a (training)
INTERVENOR (Wang 2023)	compiler errors	whole solution	iterate (2-agent)	HumanEval +18% (GPT-3.5)	**yes** (compiler)
LATS (Zhou 2024)	env + LLM value	MCTS over trajectories	tree search	HumanEval 92.7% pass@1 (GPT-4)	mixed (env + self)
AlphaCodium (Ridnik 2024)	public + AI tests	whole-flow stages	iterate fix	CodeContests pass@5 19→44% (GPT-4)	partly (real tests)

Paper	Feedback signal	Granularity	# rounds	Headline	$M_F > M_P$?
MGDebugger (Shi 2024)	LLM-simulated executor	per subfunction (tree)	hierarchical	HumanEvalFix 97.6% repair	**no** (self)
SWE-Agent (Yang 2024)	env + tests via ACI	patch / repo	autonomous loop	SWE-bench 12.5% (GPT-4)	n/a (env)
AutoCodeRover (Zhang 2024)	env + SBFL + AST search	patch	iterate	SWE-bench-lite 19%	n/a (env)
ChatRepair (Xia 2024)	test failures	conversation	multi-turn	Defects4J 162 fixes ($0.42 each)	**no** (self)
RepairAgent (Bouzenia 2024)	14 tools incl. tests	FSM-driven agent	autonomous	Defects4J 164 fixes (39 novel)	**no** (self)
LEVER (Ni 2023)	execution-aware learned verifier	candidate rerank	1 (training-time)	Spider/WikiTQ/GSM8K/MBPP +4.6–10.9%	partly (learned)
CoCoGen (Bi 2024)	compiler error types	retrieve-and-refine	iterate	CoderEval >80% relative	**yes** (compiler)

Cross-cutting observation: papers using an *external* signal (compiler, executor, retrieved repo context) — INTERVENOR, AlphaCodium, SWE-Agent, AutoCodeRover, LEVER, CoCoGen — are the ones that survive Olausson's matched-budget bar. Pure self-critique loops (Self-Refine, MGDebugger, ChatRepair, RepairAgent) match the regime Olausson predicts: gains may exist but they're modest and brittle.

Self-Refine: Iterative Refinement with Self-Feedback (Madaan et al., NeurIPS 2023)

TL;DR. LLM outputs often aren't optimal on the first try, but no general-purpose refinement loop exists without extra training. Self-Refine uses a single LLM in three roles — generator, feedback provider, refiner — looping until a stop condition, with no supervised data or RL. Across 7 tasks (dialog through code optimization) with GPT-3.5/4, Self-Refine improves outputs by ~20% absolute on average over single-shot generation.

Key mechanism. Same-model self-feedback in natural language; entirely intrinsic — no compiler, executor, or external critic.

Eval. ~13% absolute gain on code generation tasks over Codex single-shot.

SoundCode delta. Different — SoundCode's verifier is an external sound oracle (cargo+LSP), not LLM self-critique, and runs inline during decoding.

Teaching Large Language Models to Self-Debug (Chen et al., ICLR 2024)

TL;DR. Code from LLMs in a single attempt is often wrong, and prior repair methods require fine-tuning. Self-Debugging few-shot-prompts the model to execute its own code and explain it ("rubber duck"), then iterate using execution traces and (optionally) unit-test feedback. Achieves SOTA on Spider, TransCoder, and MBPP — up to +12% on TransCoder, +9% on hardest Spider queries, matching 10× sample baselines.

Key mechanism. Self-generated code explanation as feedback channel; works even without unit tests (pure explanation feedback).

Eval. TransCoder C++-to-Python: up to +12% over baseline with code-davinci-002 / gpt-3.5-turbo.

SoundCode delta. Different — feedback is post-hoc execution and self-explanation; SoundCode uses static LSP signals at structural boundaries during decoding.

Reflexion: Language Agents with Verbal Reinforcement Learning (Shinn et al., NeurIPS 2023)

TL;DR. RL fine-tuning is expensive for language agents that need to learn from sparse trial-and-error. Reflexion converts environment feedback (scalar reward / binary pass) into a free-form textual "reflection" stored in episodic memory and prepended to the next attempt, with no weight updates. Reaches 91% pass@1 on HumanEval (GPT-4), beating the prior 80% SOTA.

Key mechanism. Verbal self-reflection as cross-episode memory; reflection is generated from an external evaluator's binary/scalar reward.

Eval. HumanEval pass@1 = 91% (GPT-4 + Reflexion).

SoundCode delta. Different — Reflexion learns across full attempts post-hoc; SoundCode rolls back mid-generation to a verified prefix.

CodeT: Code Generation with Generated Tests (Chen et al., 2022)

TL;DR. Selecting the right candidate from many LLM samples is hard when test cases aren't given. CodeT prompts the same LLM to generate test cases, then uses RANSAC-style "dual execution agreement" — ranking solutions by how many tests they pass and how many similar solutions agree — to pick the best. Lifts HumanEval pass@1 from 47.0% to 65.8% on code-davinci-002, +20% over previous SOTA.

Key mechanism. Model-generated unit tests + dual consensus over (test, solution) pairs; no execution feedback fed back into generation.

Eval. HumanEval pass@1 = 65.8% (code-davinci-002 + CodeT).

SoundCode delta. Different — CodeT is post-generation reranking with stochastic tests; SoundCode is sound static verification during streaming.

CodeRL: Mastering Code Generation through Pretrained Models and Deep Reinforcement Learning (Le et al., NeurIPS 2022)

TL;DR. Standard next-token training ignores functional-correctness signals from unit tests. CodeRL casts code synthesis as actor-critic RL: the LM is fine-tuned with critic-predicted token-level returns derived from unit-test outcomes, plus a critic-guided test-time resampling-and-repair loop. Achieves SOTA on APPS and zero-shot SOTA on MBPP using a CodeT5-based backbone.

Key mechanism. Trained critic supplies dense per-token rewards; inference uses critic-filtered subsequences as seeds for repair conditioned on compiler errors.

Eval. New SOTA on APPS benchmark (CodeT5 backbone with CodeRL).

SoundCode delta. Different — CodeRL changes weights and uses dynamic tests; SoundCode is training-free and uses static LSP/compiler signals.

INTERVENOR: Prompting the Coding Ability of LLMs with the Interactive Chain of Repair (Wang et al., ACL Findings 2024)

TL;DR. LLMs suffer "Degeneration-of-Thought" when self-critiquing buggy code. INTERVENOR splits roles into a Code Learner and a Code Teacher LLM, with the Teacher consuming compiler error messages to produce a Chain-of-Repair (CoR) plan that guides the Learner. Reports ~18% / ~4.3% improvements over GPT-3.5 on code generation / translation.

Key mechanism. Two-agent decomposition with compiler errors flowing only through the Teacher; explicit natural-language repair plan as bridge.

Eval. ~18% improvement over GPT-3.5 on code generation.

SoundCode delta. Different — multi-agent post-hoc compile-fix loop; SoundCode is single-producer with in-stream LSP rollback.

Language Agent Tree Search Unifies Reasoning, Acting, and Planning (Zhou et al., ICML 2024)

TL;DR. Existing LM agents act greedily and can't backtrack or plan across alternatives. LATS adapts MCTS to language agents, using LM-generated value scores, environment feedback, and Reflexion-style self-reflection at each node to search a tree of reasoning/acting trajectories. Achieves 92.7% pass@1 on HumanEval with GPT-4, SOTA at the time.

Key mechanism. MCTS over agent trajectories with LM-as-value-function and reflective backups; first framework unifying reasoning + acting + planning + self-reflection + memory.

Eval. HumanEval pass@1 = 92.7% (GPT-4 + LATS).

SoundCode delta. Different — tree search over completed attempts; SoundCode does linear streaming with structural rollback, no branching.

Code Generation with AlphaCodium: From Prompt Engineering to Flow Engineering (Ridnik et al., 2024)

TL;DR. Single prompts and CoT plateau on competitive programming because correctness depends on subtle spec details. AlphaCodium is a hand-designed multi-stage "flow" with problem reflection, AI-generated tests, modular code, and iterative public/AI-test fixing. On CodeContests validation, lifts GPT-4 pass@5 from 19% (direct) to 44%, with ~4 orders of magnitude fewer LLM calls than AlphaCode.

Key mechanism. Engineered flow with self-generated AI tests augmenting public tests; iterative run-and-fix against both test pools.

Eval. CodeContests val pass@5 = 44% (GPT-4 + AlphaCodium) vs 19% direct.

SoundCode delta. Different — flow engineering with dynamic tests post-generation; SoundCode is static verification inline.

From Code to Correctness: Closing the Last Mile of Code Generation with Hierarchical Debugging (MGDebugger) (Shi et al., 2024)

TL;DR. Holistic LLM debugging fails to localize bugs in complex code spanning multiple granularities. MGDebugger decomposes generated code into a tree of subfunctions, generates per-subfunction tests, and debugs bottom-up using an LLM-simulated Python executor that traces variables. Improves HumanEval accuracy by up to 18.9% and reaches 97.6% on HumanEvalFix.

Key mechanism. Hierarchical subfunction decomposition with LLM-simulated execution; bottom-up bug fixing rather than holistic patching.

Eval. HumanEvalFix repair success = 97.6%; up to +18.9% over seed on HumanEval.

SoundCode delta. Different — structural decomposition is over the AST for debugging; SoundCode uses depth-0 structural boundaries for verification checkpoints.

SWE-Agent: Agent-Computer Interfaces Enable Automated Software Engineering (Yang et al., NeurIPS 2024)

TL;DR. Agents struggle to drive raw shells when solving real GitHub issues. SWE-agent introduces an Agent-Computer Interface (ACI) — purpose-built commands for file viewing/editing/search plus syntax-guardrail feedback — designed for LM ergonomics rather than humans. With GPT-4 Turbo solves 12.5% of SWE-bench (vs prior 3.8%) and 87.7% pass@1 on HumanEvalFix.

Key mechanism. Custom LM-friendly file editor with built-in syntax linter as guardrail; designed actions and feedback for LM cognitive limits.

Eval. SWE-bench resolution = 12.47% (GPT-4 Turbo + SWE-agent).

SoundCode delta. Different — repo-level agent with tool-call edits; SoundCode operates on a single streaming completion with static verification.

AutoCodeRover: Autonomous Program Improvement (Zhang et al., ISSTA 2024)

TL;DR. LLM repair agents that treat a codebase as a flat collection of files lack structural understanding. AutoCodeRover gives the LLM AST-aware code-search APIs (search_class, search_method_in_file, etc.) and optionally spectrum-based fault localization to iteratively retrieve context before patch synthesis. Resolves 19% of SWE-bench-lite in ~4 minutes per issue at $0.43 average cost.

Key mechanism. AST-grounded code search APIs + SBFL-guided context retrieval; tree-structured program representation rather than flat files.

Eval. SWE-bench-lite resolution = 19% with AutoCodeRover.

SoundCode delta. Different — repo-search APR agent; SoundCode does not perform retrieval or multi-file edits.

Automated Program Repair via Conversation (ChatRepair) (Xia & Zhang, ISSTA 2024)

TL;DR. Prior LLM-APR resamples i.i.d. from the same prompt, ignoring test-failure semantics and prior failed/plausible patches. ChatRepair feeds ChatGPT relevant test-failure info as starting context, then conversationally builds on both failures and plausible patches across turns to drive diversity. Fixes 162/337 Defects4J bugs at $0.42 each; new SOTA at 114 (v1.2) and 48 (v2.0) correct fixes.

Key mechanism. Stateful multi-turn dialogue carrying both negative (failure) and positive (plausible) patch context — not i.i.d. resampling.

Eval. Defects4J v1.2 / v2.0: 114 / 48 correct fixes (ChatGPT + ChatRepair).

SoundCode delta. Different — post-generation conversational APR using dynamic tests; SoundCode acts pre-completion with static signals.

RepairAgent: An Autonomous, LLM-Based Agent for Program Repair (Bouzenia et al., ICSE 2025)

TL;DR. Prior LLM repair loops are hard-coded and don't let the model choose what info to gather. RepairAgent equips GPT-3.5 with 14 bug-fixing tools (read code, search, run tests, propose/validate patches) plus a finite-state-machine middleware with a dynamically updated prompt, letting the LLM plan tool invocations autonomously. Fixes 164/835 Defects4J bugs, including 39 unfixed by prior work, at ~14¢ per bug.

Key mechanism. Autonomous tool-using agent with FSM-guided dynamic prompt; first APR work to let the LLM plan tool sequences.

Eval. Defects4J: 164 bugs fixed (74 in v1.2, 90 in v2.0), 39 novel vs prior SOTA.

SoundCode delta. Different — tool-using post-hoc APR agent; SoundCode is a streaming generator with sound static feedback, no tool selection by the LLM.

LEVER: Learning to Verify Language-to-Code Generation with Execution (Ni et al., ICML 2023)

TL;DR. Filtering by execution errors / majority vote ignores rich semantic signal in execution results. LEVER trains a verifier on (NL, program, execution-result) triples to score candidates, then reranks by joint verifier × generator probability marginalized over programs with identical execution. Improves code-davinci-002 by 4.6–10.9% across Spider, WikiTQ, GSM8k, MBPP — SOTA on all four.

Key mechanism. Learned verifier over execution traces (not just error/no-error); separately trained reranker module.

Eval. +4.6% to +10.9% over code-davinci-002 across four datasets; new SOTA.

SoundCode delta. Different — trained probabilistic verifier as post-hoc reranker; SoundCode uses sound symbolic verifier inline.

Iterative Refinement of Project-Level Code Context for Precise Code Generation with Compiler Feedback (CoCoGen) (Bi et al., ACL Findings 2024)

TL;DR. Repo-level code generation fails because LLMs lack project-specific APIs/types and can't fit the whole codebase in context. CoCoGen compiles each generated snippet, uses static analysis to identify context mismatches (UNDEF, API, OBJECT errors), then retrieves targeted repo context to repair and re-generate iteratively. Improves vanilla GPT-3.5-Turbo and Code Llama 13B by >80% relative on CoderEval project-level pass rates.

Key mechanism. Compiler-error-typed retrieval loop — compile, classify error, fetch project context, refine — rather than blind RAG.

Eval. CoderEval project-level: >80% relative improvement over vanilla LLMs and prior retrieval baselines.

SoundCode delta. Closest of the iterative-compiler family, but different — CoCoGen recompiles whole snippets between attempts; SoundCode invokes cargo+LSP asynchronously at boundaries during streaming and rolls back rather than re-prompting.

---

3. Benchmarks — fast cards

The ten benchmarks below are the candidate evaluation sets for SoundCode. The key columns to watch are **language** (only the Rust-native ones — RustEvo², CRUST-Bench, RustRepoTrans,

and Rust slices of MultiPL-E — are directly usable; the rest require translation or are out-of-scope) and **contamination-safe** (only LiveCodeBench and the newer 2025 Rust benchmarks are designed to escape training-cutoff contamination).

3.0 Benchmarks comparison table

Name	#Problems	Language	Tests/problem	Contamination-safe	Year	Usable by SoundCode?
HumanEval	164	Python	~7.7	**no** (in every code corpus)	2021	only via MultiPL-E Rust port
MultiPL-E	164 + 974	19 langs incl. **Rust**	translated from Py	partial (Rust port less memorized)	2023	**YES** (HumanEval-rs, MBPP-rs)
LiveCodeBench	500+ (growing)	Python	varies	**YES** (date-windowed)	2024	methodology only (no Rust)
BigCodeBench	1,140	Python	~5.6	no	2025	no (Python library-call heavy)
RustEvo²	588	**Rust**	varies	partial (API-evolution focus)	2025	**YES** (native Rust)
CRUST-Bench	100 repos	C → **Rust** (multi-file)	suite per repo	partial	2025	**YES** (native Rust, multi-file)
RustRepoTrans	375	C/Java/Python → **Rust**	varies	partial	2025	**YES** (repo-aware Rust)
RepoBench	1,669 repos	Python/Java	n/a (next-line)	partial (post-cutoff crawl)	2023	no
CrossCodeEval	~10k	Py/Java/TS/C#	n/a (next-line)	partial	2023	no (no Rust split)
CRUXEval	800	Python	1 assertion	partial	2024	no (reasoning, not generation)

Evaluating Large Language Models Trained on Code (Chen et al., 2021)

TL;DR. Introduces Codex and the HumanEval benchmark to measure functional correctness of docstring-to-function synthesis, replacing match-based metrics with pass@k. Each problem is a hand-written Python function with a docstring, signature, and hidden unit tests. Headline scale: 164 problems with ~7.7 unit tests per problem on average; Codex-12B solves 28.8% at pass@1.

Format. `(docstring_prompt, function_signature, hidden_unit_tests, language=Python)`.

Size + language. 164 problems, Python only.

SoundCode delta. Not Rust-native; only usable via MultiPL-E's HumanEval-Rust port. Highly contaminated by 2024+ (seven-figure citation count, in nearly every code corpus), so SoundCode should treat it as a sanity check, not a headline metric.

MultiPL-E: A Scalable and Polyglot Approach to Benchmarking Neural Code Generation (Cassano et al., IEEE TSE 2023)

TL;DR. Closes the polyglot gap by translating Python NL2Code benchmarks (HumanEval, MBPP) into 18 additional languages via small per-language compilers that rewrite signatures, doctests, unit tests, and type annotations. Produces parallel multi-language problem sets so a single problem can be scored across languages with hidden unit tests. Empirically shows that code LMs often match or exceed Python performance on JavaScript/C++/Scala/TypeScript and that perplexity is a poor predictor of correctness.

Format. `(translated_prompt, translated_signature, translated_hidden_tests, target_language ∈ 19 langs)`.

Size + language. HumanEval (164) + MBPP (~974) translated into 18 additional languages (19 total, including Rust).

SoundCode delta. Directly usable: MultiPL-E ships a Rust translation of HumanEval/MBPP, the standard "easy" Rust functional-correctness benchmark. Contamination risk is high since it derives from HumanEval/MBPP, but Rust translations are less memorized than Python originals.

LiveCodeBench: Holistic and Contamination Free Evaluation of LLMs for Code (Jain et al., 2024)

TL;DR. Addresses contamination and narrow scope of HumanEval/MBPP by collecting new contest problems over time and tagging each with a release date so evaluators can window to post-cutoff problems only. Sources problems from LeetCode, AtCoder, and CodeForces and adds three auxiliary scenarios (self-repair, code execution, test-output prediction) beyond generation. Hosts 500+ problems released between May 2023 and May 2024 at the time of writing; demonstrates DeepSeek/GPT-4-O performance drops sharply on problems after their training cutoff.

Format. `(problem_statement, starter_code_or_signature, hidden_tests, release_date, language=Python)` plus self-repair/execution/test-prediction variants.

Size + language. 500+ problems (growing), Python only.

SoundCode delta. Not Rust — Python-only contest problems. Contamination-safe by construction (date-windowed). For SoundCode, mainly a methodological reference for the contamination-window protocol, not a benchmark to run.

BigCodeBench: Benchmarking Code Generation with Diverse Function Calls and Complex Instructions (Zhuo et al., ICLR 2025)

TL;DR. Targets the gap between toy algorithmic problems and realistic library-heavy code by requiring multi-library, multi-tool function-call programs from complex instructions. Each task pairs a structured docstring (or an NL-rewritten "Instruct" variant) with a rigorous `unittest`-based test suite averaging 5.6 tests and 99% branch coverage. 1,140 tasks span 723 function calls across 139 libraries in 7 domains; best model (GPT-4o) reaches only 60% on Complete and <50% on Instruct, vs. 97% human.

Format. `(structured_docstring_or_NL_instruction, function_signature, unittest_class_with_~5.6_t` `language=Python)` with two variants: Complete and Instruct.

Size + language. 1,140 tasks, Python only.

SoundCode delta. Not Rust — Python-library-call-heavy. Library-call orientation does not translate to Rust crates, so SoundCode cannot use it directly; cite as evidence that "functional correctness" benchmarks should include realistic API use.

RustEvo²: An Evolving Benchmark for API Evolution in LLM-based Rust Code Generation (Liang et al., 2025)

TL;DR. Constructs the first Rust benchmark that explicitly tests API-evolution awareness, since Rust ships a new minor version roughly every six weeks and LLMs lag the ecosystem. Automates dataset construction by mining API changes (stabilizations, signature changes, behavioral changes, deprecations) from Rust std and crates, then having LLMs synthesize tasks that implicitly require the new APIs with executable Rust tests. Builds 588 API-change tasks (380 std + 208 third-party from 15 crates) spanning Rust 1.71.0–1.84.0; SOTA models average 65.8% pass on stabilized APIs but drop to 38.0% on behavioral changes.

Format. `(NL_description, function_signature, executable_Rust_tests, required_API_version, change_category)`.

Size + language. 588 tasks, Rust only (4 API-change categories across std and 15 crates).

SoundCode delta. Directly Rust-native and a natural fit for SoundCode — LSP signal is especially valuable when models drift toward deprecated/old-signature APIs. Built from post-2023 Rust versions, so contamination risk is moderate and tilted toward older APIs.

CRUST-Bench: A Comprehensive Benchmark for C-to-safe-Rust Transpilation (Khatry et al., 2025)

TL;DR. Addresses the absence of a multi-file, safety-aware C-to-Rust transpilation benchmark by curating 100 C repositories each paired with a hand-written safe-Rust interface (signatures, types, ownership) plus a test suite. Tasks require whole-repository translation into idiomatic, memory-safe Rust that compiles and passes the provided tests. Best model (OpenAI o3) solves only 19/100 single-shot, rising to 32–48% with a repair loop; CRUST-Bench averages 958 LoC per project across multiple files.

Format. `(C_repo, safe_Rust_interface_spec, Rust_test_cases, multi_file=True)`.

Size + language. 100 C repositories → safe Rust (multi-file, avg 958 LoC).

SoundCode delta. Directly Rust-native; an excellent stress test for LSP-supervised generation since Rust's borrow checker and type system carry most of the "safe transpilation" load. Released 2025 from curated repos — relatively contamination-safe.

RustRepoTrans: Repository-level Code Translation Benchmark Targeting Rust (Ou et al., ASE 2025)

TL;DR. Fills the gap between function-level and full-repo translation benchmarks by curating repository-level *incremental* translation tasks where dependencies, cross-module references, and architectural divergence matter. Each task is a source-function/target-function pair with its required dependencies (functions, types, variables, libraries) and Rust test cases extracted from real GitHub projects that exist in both source and Rust form. 375 tasks translating from C/Java/Python into Rust; DeepSeek-R1 leads at 51.5% Pass@1 and drops 22.2 points (73.7% → 51.5%) when repo context is added.

Format. (source_function, target_signature, target_dependencies, Rust_test_cases, source_lang ∈ {C, Java, Python}).

Size + language. 375 tasks, target = Rust (source = C/Java/Python).

SoundCode delta. Directly Rust-native and repo-aware — a strong fit for LSP-supervised SoundCode since cross-module type lookups are exactly what the LSP provides. Built from real open-source rewrites; contamination risk depends on rewrite age but moderate overall.

RepoBench: Benchmarking Repository-Level Code Auto-Completion Systems (Liu et al., 2023)

TL;DR. Targets repo-level code auto-completion to escape single-file evaluations and introduces three connected sub-tasks (Retrieval, Completion, Pipeline) reflecting a Copilot-like workflow. Test repos are newly crawled GitHub Python/Java projects created Feb–Aug 2023 to mitigate leakage against The Stack's cutoff. 1,075 Python + 594 Java test repositories with cross-file-first/random/in-file settings; metric is next-line accuracy.

Format. (in_file_context, cross_file_context_snippets, cursor_position, gold_next_line, language ∈ {Python, Java}).

Size + language. 1,075 Python + 594 Java test repos; 2 languages.

SoundCode delta. Not Rust — Python/Java only. Useful as a comparison point for the "repo-level completion + retrieval" design pattern, but SoundCode cannot evaluate on it directly.

CrossCodeEval: A Diverse and Multilingual Benchmark for Cross-File Code Completion (Ding et al., 2023)

TL;DR. Builds a cross-file completion benchmark where each example *strictly requires* cross-file context, by using static analysis to find call sites whose targets become undefined when imports are stubbed. Spans four languages and verifies via exact match and identifier-F1 against gold next tokens. 10k examples drawn from 1k repos (471 Py, 239 Java, 193 TS, 99 C#) created after mid-2023 to avoid Stack-era leakage; cross-file context boosts CodeGen/StarCoder substantially but ceiling remains far from solved.

Format. (in_file_prefix, cross_file_context, cursor_position, gold_completion, language ∈ {Python, Java, TypeScript, C#}).

Size + language. ~10k examples across 1k repos, 4 languages (no Rust).

SoundCode delta. Not Rust — no Rust split, and the static-analysis pipeline (Pylint/javac/tsc) does not cover Rust. Cite as the canonical "cross-file context required" benchmark; SoundCode would need to reproduce the pipeline with rust-analyzer for a Rust analog.

CRUXEval: A Benchmark for Code Reasoning, Understanding and Execution (Gu et al., 2024)

TL;DR. Probes code *reasoning* rather than generation by asking models to predict program behavior on short Python functions in two directions: input prediction (CRUXEval-I) and output prediction (CRUXEval-O). Constructed via filtered Code-Llama-34B generation to keep simple, human-solvable functions. 800 Python functions (3–13 lines), each with an `assert f(input) == output` pair; GPT-4 reaches 67% / 63% (I/O); CoT helps but no model "aces" it.

Format. (short_Python_function, input, output) used for either input-prediction or output-prediction tasks.

Size + language. 800 functions, Python only.

SoundCode delta. Not Rust — Python execution-reasoning probe, not a code-generation benchmark. Out of scope for SoundCode's generation pipeline; relevant only as conceptual evidence that HumanEval-style pass@k underspecifies "understanding."

---

4. Code-LM technical reports — fast cards

The three open-weight code-LM families below are the candidate generators for SoundCode's model grid. Qwen2.5-Coder is the primary (already exercised in week-5 runs); StarCoder 2 and DeepSeek-Coder-V2 are open candidates if generalization across families becomes a concern.

4.0 Model-reports comparison table

Model	Sizes	HumanEval pass@1 (best)	Released	In SoundCode grid?
Qwen2.5-Coder	0.5B / 1.5B / 3B / 7B / 14B / 32B (base + instruct)	65.9% (32B base, greedy)	2024	**YES** (primary; already in `results/qwen2.5-coder_32b_*.json`)
StarCoder 2	3B / 7B / 15B (base only)	46.3% (15B)	2024	candidate — strong Rust on MultiPL-E (15B = 38.0), full data transparency via The Stack v2
DeepSeek-Coder-V2	16B-Lite (2.4B active) / 236B (21B active) MoE	90.2% (236B instruct)	2024	candidate — frontier-open; 338-lang code corpus implies non-trivial Rust

Qwen2.5-Coder (Hui et al., 2024)

TL;DR. Code-specialized LLM family derived from Qwen2.5, continuously pretrained on a coding-focused corpus. Six base + instruct sizes (0.5B–32B) trained on over 5.5 trillion tokens of code-heavy data. Qwen2.5-Coder-32B-Base reaches 65.9% HumanEval pass@1, claimed SOTA among open code models at its size and rivaling GPT-4o for the Instruct variant.

Sizes. 0.5B, 1.5B, 3B, 7B, 14B, 32B (base and instruct).

HumanEval pass@1. 65.9% (Qwen2.5-Coder-32B base, greedy; Table 5).

SoundCode delta. Primary generator family for SoundCode's model grid (already in `results/qwen2.5-coder_32b_*` runs); paper does not break down Rust share of pretraining mix.

StarCoder 2 (Lozhkov et al., 2024)

TL;DR. Fully open BigCode/SWH code LLM trained on The Stack v2, spanning 619 programming languages. Three base sizes (3B/7B/15B) trained on 3.3–4.3 trillion tokens, with OpenRAIL weights and full data transparency via SWHIDs. StarCoder2-15B reaches 46.3% Hu-

manEval pass@1 and beats DeepSeek-Coder-33B on low-resource languages and code reasoning.

Sizes. 3B, 7B, 15B (base only).

HumanEval pass@1. 46.3% (StarCoder2-15B, greedy; Table 9).

SoundCode delta. Plausible secondary generator for SoundCode given strong Rust coverage on MultiPL-E (StarCoder2-15B scores 38.0 on Rust, Table 10) and full data transparency for reproducibility.

DeepSeek-Coder-V2 (Zhu et al., 2024)

TL;DR. Open-source Mixture-of-Experts code LLM continued from DeepSeek-V2 with an additional 6T tokens (60% code, 10% math, 30% NL). Two MoE sizes with 2.4B/21B active parameters, 338 programming languages, and 128K context. DeepSeek-Coder-V2 (236B total / 21B active) achieves 90.2% HumanEval pass@1 and surpasses GPT-4-Turbo, Claude 3 Opus, and Gemini 1.5 Pro on coding benchmarks.

Sizes. 16B (2.4B active) Lite, 236B (21B active) full; base and instruct.

HumanEval pass@1. 90.2% (DeepSeek-Coder-V2 236B instruct; Section 1.2).

SoundCode delta. Strong frontier-open candidate for SoundCode's top-tier generator slot; 338-language code corpus implies non-trivial Rust coverage, though paper does not give per-language pretraining percentages.

---

5. Methodology / critique — fast cards

Two papers that establish the methodological bar SoundCode evaluations must clear. Out-of-the-BLEU rules out using n-gram metrics to claim wins over baselines; Statistical Precipice rules out comparing point estimates across few seeds. Together with Olausson (Section 1.D) they constrain *how* SoundCode reports any improvement number.

Out of the BLEU: How Should We Assess Quality of the Code Generation Models? (Evtikhiev et al., 2023)

TL;DR. BLEU and CodeBLEU were imported from MT without validation for code generation, yet researchers routinely claim superiority from tiny score gaps. The authors run a 6-metric (BLEU, ROUGE-L, METEOR, ChrF, CodeBLEU, RUBY) study against human grades on CoNaLa and HearthStone, and recommend bootstrap significance testing plus migrating away from BLEU. Headline: on CoNaLa no metric reliably distinguishes models within <5 points, and ChrF (and ROUGE-L) align with humans better than BLEU/CodeBLEU.

Critique target. Using small BLEU/CodeBLEU deltas to claim one code-generation model beats another.

Recommended replacement. Report ChrF (or test-based pass rates) with paired bootstrap resampling for corpus-level significance, and refuse to claim wins below the metric's noise floor (~2–5 points).

SoundCode delta. When SoundCode reports per-model quality, use execution/compilation pass rates as primary and any n-gram-style metric only with bootstrap CIs, never claiming a win on small absolute differences.

Deep Reinforcement Learning at the Edge of the Statistical Precipice (Agarwal et al., NeurIPS 2021)

TL;DR. Deep RL benchmarks routinely report point estimates of mean/median over a handful of runs, hiding statistical uncertainty and producing irreproducible rankings. The authors prescribe stratified bootstrap confidence intervals, performance profiles, and the interquartile mean (IQM) as a robust aggregate, packaged in the `rliable` library. Re-analyzing Atari 100k, ALE, Procgen, and DM Control with these tools surfaces substantial discrepancies versus prior published comparisons.

Critique target. Reporting mean/median point estimates over few seeds and declaring SOTA without uncertainty quantification.

Recommended replacement. Stratified bootstrap CIs on IQM, paired with performance profiles and probability-of-improvement / optimality-gap for pairwise claims.

SoundCode delta. Across the per-benchmark/per-model grid, report IQM of pass@k (or latency-bounded pass rate) with stratified bootstrap 95% CIs and probability-of-improvement vs. the baseline decoder, rather than single-seed means.

---

6. SoundCode positioning

Paste-ready paragraphs for the related-work section of the eventual paper. One per category. Numbers are TBD — the structure is what's stable.

6.1 vs. ROCODE / SemGuard / IterGen (decoding-time *with* rollback)

> The closest neighbors are ROCODE (Jiang et al., 2024) and SemGuard (Wang et al., 2025), both of which perform mid-stream rollback when a verifier signals that the partial program is wrong. ROCODE uses a compiler as the verifier and rolls back at statement boundaries with an exponentially-decaying token penalty; SemGuard trains a 1.3B learned semantic evaluator and rolls back at line boundaries with a single-token penalty plus resample. Both verifiers run *synchronously* on the decode critical path — practical for Python and Java where the verifier costs tens of milliseconds. **SoundCode targets Rust, where `cargo check` and rust-analyzer cost 100 ms to several seconds.** This forces a different design: we run the verifier *asynchronously* alongside token streaming, and trigger rollback at depth-0 `;/}` structural boundaries to the last verified checkpoint, with no learned component. The async + real-compiler combination is the open design point ROCODE explicitly defers ("we focus on compilers that are fast enough to call synchronously"). IterGen (Ugare et al., ICLR 2025) supports rollback at grammar-non-terminal boundaries using KV-cache cropping, but its verifier is a user-supplied predicate over a CFG — the technique applies to structured DSLs (SQL, Vega-Lite) rather than Turing-complete languages with whole-crate type and lifetime constraints.

6.2 vs. MGD / Type-Constrained / PICARD / Synchromesh / GCD / DOMINO (decoding-time *without* rollback)

> A parallel line of work — Monitor-Guided Decoding (Agrawal et al., NeurIPS 2023), Type-Constrained Decoding (Mündler et al., PLDI 2025), PICARD (Scholak et al., EMNLP 2021), Synchromesh (Poesia et al., ICLR 2022), Grammar-Constrained Decoding (Geng et al., 2023), and DOMINO (Beurer-Kellner et al., ICML 2024) — runs a verifier per-token and masks out invalid candidates so the LM never emits a bad prefix. This is *prevention*; SoundCode is *recovery*. Per-token masking is sound only

when the verifier is decidable on partial code (CFG membership, prefix-typedness), and it cannot catch errors that surface only when a structural unit completes — borrow violations, trait-resolution failures, multi-statement type errors. SoundCode accepts that bad prefixes will be drafted and recovers by rollback; this catches the multi-token error classes that no per-token mask can prevent, at the cost of speculation-then-rollback overhead which the async design amortizes. MGD is the strongest direct ancestor — it uses the same LSP plumbing (multilspy) and the same "wait state → trigger → constraint" state machine — but pays a reported 83% slowdown for its synchronous per-trigger LSP round-trip.

6.3 vs. Speculative-decoding family (structural blueprint)

SoundCode's control flow — draft N tokens, verify in parallel, accept or roll back — is structurally identical to Speculative Decoding (Leviathan et al., ICML 2023), Speculative Sampling (Chen et al., 2023), and Lookahead Decoding (Fu et al., 2024). The difference is the verifier. In all three of those papers the verifier is the target LLM itself (Leviathan/Chen) or the same LLM under Jacobi iteration (Fu), so the rollback signal is *probabilistic* (the target's next-token distribution disagrees with the draft) and the correctness guarantee is distributional (the output equals what the target alone would have produced). SoundCode replaces this with a *semantic* verifier: the rollback signal is a blocking diagnostic from `cargo check` or rust-analyzer, and the correctness guarantee is type-and-borrow soundness at the last verified prefix. The structural-rollback unit at depth-0 `;/}` plays the role of the speculative-decoding window γ, but is grammatically anchored rather than fixed-width — so the SpS-style α-vs-γ trade-off becomes a checkpoint-spacing question.

6.4 vs. Olausson (methodological cudgel)

Olausson et al. (ICLR 2024) showed that under matched-budget pass@k, LLM self-repair only beats plain i.i.d. resampling when the feedback model is strictly stronger than the generator — same-model self-critique cancels its own cost. SoundCode is engineered to fall into the regime Olausson predicts repair *does* win: the Rust compiler and rust-analyzer are sound oracles on the decidable fragment of correctness (type errors, borrow errors, name resolution) that they cover, so they are *strictly* stronger than any LLM on those queries. To honor Olausson's protocol, SoundCode evaluations report (a) matched-token i.i.d. resampling as a primary baseline, (b) per-difficulty breakdowns isolating cases where the verifier intervenes, and (c) ablations on the verifier itself (cargo only vs. cargo + LSP) — the analog of Olausson's M_F ablation.

6.5 vs. iterative-repair family

The iterative-repair family (Self-Refine, Self-Debug, Reflexion, INTERVENOR, LATS, AlphaCodium, MGDebugger, SWE-Agent, AutoCodeRover, ChatRepair, RepairAgent, LEVER, CoCoGen) places the verifier off the critical path — generate a complete attempt, check, revise. The recurring pattern in this family is that gains come from external signals (compilers, executors, retrieved repo context) and rarely from self-critique alone, consistent with Olausson's finding. SoundCode moves the verifier *onto* the critical path while keeping it asynchronous, so it pays roughly no extra inference cost beyond a single forward pass; the cost it pays is rollback frequency, which is bounded by checkpoint granularity. The closest comparator in this family is CoCoGen (Bi et al., 2024), which also uses compiler errors to drive refinement — but recompiles whole snippets between attempts rather than running the verifier at structural boundaries during generation.

6.6 The open design point

> Across all four neighbouring families, the **async × real-compiler × structural-rollback × Rust** combination is unaddressed by prior work. Each axis is individually motivated: async because Rust's verifier cost is too high to block on synchronously; real-compiler because Rust's borrow checker and trait resolution are exactly the errors learned evaluators and grammars cannot catch; structural-rollback because per-token masking is not sound for multi-token semantic errors; Rust because it is the language where this combination is forced (Python and C++ verifiers are too fast to bother with async; Java's JVM startup cost makes it possible but less acute). The four-mode reasoning-model orchestration matrix (R0 / R1 / T2 / C3, week-5 contribution) is an additional axis no surveyed paper studies. The defensible publication claim is the combination of (1) async LSP-supervised decoding, (2) structural rollback policy, and (3) the verifier-cost-spectrum framing that makes the trade-off quantitative — with ROCODE on Rust as the matched-mechanism baseline (Section 5.4 of the project plan).

End of related-work review. Last updated 2026-05-24. Source PDFs in `lit-rev/papers/`; the inner-ring agents that drafted Section 1 are recorded in conversation history; outer-ring cards in Sections 2–5 were drafted by per-category synthesis agents.